

Smart-Invest AWS Architecture — Interview Q&A

架构面试题（英文 + 中文）

面向英文面试，英语词汇控制在高中水平。

数据中心：新加坡（ap-southeast-1）主站 + 香港（ap-east-1）灾备。

Part 1 — JD Core Skills（Java 21 · OCP · Kafka · DDD · TDD）

第一部分：JD 核心技术栈

以下题目直接对应岗位 JD 技术要求。

Q1. What is new in Java 21 that you find most useful? Tell me about Virtual Threads.

问题1：Java 21 最有用的新特性是什么？请介绍虚拟线程。

English Answer:

Java 21 has many new features. The most important one for a server-side system is **Virtual Threads** (from Project Loom).

Old way — Platform Threads: A normal Java thread maps 1-to-1 to an OS thread. OS threads are expensive — creating 10,000 threads uses a lot of memory and the OS spends a lot of time switching between them. So Spring Boot uses a thread pool with maybe 200 threads. If 200 requests are all waiting for a database response, the 201st request must wait.

New way — Virtual Threads: Virtual threads are very lightweight. You can create **millions** of them. When a virtual thread is blocked (waiting for a database or network call), Java automatically parks it and lets another virtual thread run on the same OS thread. This is called **non-blocking I/O without changing your code**.

```
// Spring Boot 3 – enable virtual threads in application.properties
spring.threads.virtual.enabled=true
// That's it. No code changes needed.
```

Other useful Java 21 features:

- **Record classes** — simple data holders, no boilerplate getters/setters
- **Pattern matching for switch** — cleaner if/else logic
- **Sealed classes** — control which classes can extend a type (useful for DDD value objects)

中文解释：

Java 21 最重要的特性是**虚拟线程（Virtual Threads）**：

- **旧方式**：平台线程（OS线程），创建成本高，线程池上限约200个，一旦全部阻塞在等I/O，新请求只能排队。
- **虚拟线程**：极轻量级，可创建数百万个。线程阻塞时自动让出OS线程，等待完成后继续执行，本质上是在JVM层实现了协程。Spring Boot 3 只需一行配置即可启用，无需改业务代码。

其他实用特性：Record类（简化DTO/Value Object）、Switch模式匹配（简化条件分支）、Sealed类（配合DDD封闭类型体系）。

Q2. What are the main differences between Spring Boot 2 and Spring Boot 3?

问题2：Spring Boot 2 和 Spring Boot 3 的主要区别是什么？

English Answer:

Spring Boot 3 made three big changes:

1. **Jakarta EE 9+ (namespace change)** All `javax.*` packages are renamed to `jakarta.*`. This is the most common breaking change when migrating. For example:
`javax.persistence.Entity` → `jakarta.persistence.Entity`. If your code uses Spring Security, JPA, or Servlet APIs, you must update all imports.
2. **Minimum Java version is Java 17** Spring Boot 2 supports Java 8. Spring Boot 3 requires Java 17 at minimum. In practice, we use Java 21 for virtual threads.

3. **Better observability out of the box** Spring Boot 3 uses **Micrometer** with a new `@Observed` annotation. You can add distributed tracing (X-Ray, Zipkin) and metrics with almost no code. It also supports **native compilation** with GraalVM — the app starts in milliseconds instead of seconds.

中文解释：

Spring Boot 3 三个核心变化：

1. **javax → jakarta 命名空间迁移**：所有 `javax.*` 包改为 `jakarta.*`，这是升级时最常见的编译报错，批量替换import即可。
2. **最低Java 17**：Spring Boot 2支持Java 8，Spring Boot 3要求Java 17+，搭配Java 21可使用虚拟线程。
3. **可观测性增强**：内置Micrometer Tracing，一个 `@Observed` 注解自动生成链路追踪span；支持GraalVM原生镜像，启动时间从秒级降到毫秒级（对Kubernetes弹性伸缩非常有价值）。

Q3. What is OCP (OpenShift)? How is it different from plain Kubernetes?

问题3：什么是 OCP（OpenShift）？和原生 Kubernetes 有什么区别？

English Answer:

OCP stands for **OpenShift Container Platform**. It is Red Hat's version of Kubernetes. Think of it as "Kubernetes + many extra enterprise features already set up."

Key differences:

Feature	Kubernetes (plain)	OCP (OpenShift)
Security	You set it up yourself	Very strict by default (no root containers, SCC policies)
Ingress	Ingress resource	Route resource (similar, but OCP-specific)
Image registry	External (ECR, etc.)	Built-in internal registry
Web console	Basic dashboard	Full web console with build pipelines
Developer workflow	<code>kubectl apply</code>	<code>oc new-app</code> , built-in Source-to-Image (S2I)
Support	Community	Red Hat enterprise support

In practice, if your company uses OCP instead of plain EKS, the main things to learn are:

- Use `oc` command instead of `kubectl` (most commands are the same)
- Use **Routes** instead of Ingress
- Watch out for **Security Context Constraints (SCC)** — OCP blocks root containers by default

中文解释：

OCP = OpenShift Container Platform，是Red Hat基于Kubernetes的企业发行版，相当于"Kubernetes + 企业级预装功能套件"。

关键差异：

- **安全性**：OCP默认禁止root容器运行（SCC策略），比原生K8s严格得多。迁移时最容易踩坑：镜像里以root启动的服务在OCP上直接报错。
- **Route vs Ingress**：OCP用Route替代Ingress，功能类似但语法不同。
- **内置镜像仓库**：OCP自带镜像仓库，原生K8s需要自己对接ECR/Harbor等。
- **命令行**：`oc` 命令，大部分与 `kubectl` 相同，额外提供 `oc new-app` 等开发者友好命令。

面试回答要点：理解两者本质相同（都是K8s），OCP加了安全合规层，在金融/政府场景更受欢迎。

Q4. What is the difference between Kafka and IBM-MQ? When do you use each?

问题4: Kafka 和 IBM-MQ 有什么区别? 什么场景用哪个?

English Answer:

Both Kafka and IBM-MQ are message systems. They solve different problems.

IBM-MQ — Traditional Message Queue:

- Each message is delivered to **exactly one consumer**. Once consumed, it is gone.
- Best for: "send a payment request, one service processes it, done." The message must not be processed twice.
- Guarantees: **at-most-once** or **exactly-once** delivery.
- Used in banking for many years. Very reliable. Enterprise support.

Kafka — Event Streaming Platform:

- Messages are stored as a **log** and can be read by **many consumers independently**.
- Best for: "a user made a trade — the risk service, the audit service, and the portfolio service all need to know about it." Each service reads the same event at its own pace.
- Messages are kept for days or weeks (configurable). You can replay old events.
- Much higher throughput than IBM-MQ.

Simple rule:

- Use **IBM-MQ** when you need guaranteed exactly-once processing of commands (e.g., "transfer money").
- Use **Kafka** when you need to broadcast events to many services, or need to replay events (e.g., "user logged in" — audit, analytics, fraud detection all care).

中文解释:

	IBM-MQ	Kafka
消息消费	一条消息只被一个消费者处理（点对点）	一条消息可被多个消费者组独立消费
消息保留	消费后删除	保留数天/数周，可重放
吞吐量	中等	极高（百万级/秒）
适用场景	命令（精确一次执行）：转账、下单	事件（广播）：用户行为、审计日志、风控

面试答法：IBM-MQ用于"命令"（需要精确一次处理，不能重复），Kafka用于"事件"（需要多个服务响应同一事件，或需要事件回溯）。两者在同一系统中可以共存。

Q5. What is Domain-Driven Design (DDD)? Explain with an example from a financial system.

问题5：什么是领域驱动设计（DDD）？请用金融系统举例说明。

English Answer:

DDD is a way to design software by focusing on the **business domain** — the real-world problem you are solving. The code structure should reflect how the business works, not how the database looks.

Key concepts:

1. **Bounded Context** — a clear boundary around one part of the business. In our system:
 - **Order Context** : handles placing and managing orders
 - **Portfolio Context** : handles what assets a user holds
 - **Fund Context** : handles fund information Each context has its own database and does not share tables with others.
2. **Aggregate** — a cluster of objects that must stay consistent together. For example, an **Order** aggregate contains the **Order** itself plus its **OrderItems** . You always save or load them together. The **Order** is the **Aggregate Root** — the only entry point to change the data.

3. **Domain Event** — something important that happened in the business. For example: `OrderPlaced`, `FundPurchased`, `PortfolioUpdated`. Other bounded contexts listen for these events (via Kafka) and react.
4. **Value Object** — an object defined by its value, not its identity. For example, `Money(100, USD)` — two `Money` objects with the same amount and currency are equal. No ID needed. Use Java `record` classes for these.

中文解释：

DDD核心思想：代码结构反映业务结构，而不是反映数据库表结构。

金融系统中的应用：

- **限界上下文（Bounded Context）**：order-service、fund-service、portfolio-service各自是独立的上下文，各自拥有自己的数据库，互不直接访问对方的表。
- **聚合（Aggregate）**：`Order` 聚合包含Order本身 + 若干OrderItem，只能通过 `Order`（聚合根）修改，保证一致性。
- **领域事件（Domain Event）**：`OrderPlaced` 事件通过Kafka发布，portfolio-service监听后更新持仓，实现解耦。
- **值对象（Value Object）**：`Money(100, "USD")`，用Java 21 Record实现，两个Money只要金额和币种相同就相等，不需要ID。

Q6. What is TDD? Show me the Red-Green-Refactor cycle with a simple example.

问题6：什么是 TDD？请用一个简单例子演示红-绿-重构循环。

English Answer:

TDD (Test-Driven Development) means you write the test **before** you write the code. The cycle has three steps:

1. **Red** — write a test that fails (because the code doesn't exist yet)
2. **Green** — write the minimum code to make the test pass
3. **Refactor** — clean up the code, keeping the test green

Example — calculate order total with discount:

```
// Step 1: RED – write the test first (it fails because OrderService doesn't
// exist yet)
@Test
void shouldApply10PercentDiscountWhenOrderOver1000() {
    OrderService service = new OrderService();
    Money total = service.calculateTotal(new BigDecimal("1200"));
    assertThat(total.getAmount()).isEqualByComparingTo("1080.00"); // 1200 *
    0.9
}

// Step 2: GREEN – write just enough code to pass
public class OrderService {
    public Money calculateTotal(BigDecimal amount) {
        if (amount.compareTo(new BigDecimal("1000")) > 0) {
            return new Money(amount.multiply(new BigDecimal("0.9")));
        }
        return new Money(amount);
    }
}

// Step 3: REFACTOR – extract the discount threshold to a named constant
private static final BigDecimal DISCOUNT_THRESHOLD = new BigDecimal("1000");
private static final BigDecimal DISCOUNT_RATE = new BigDecimal("0.9");
```

Why TDD matters for this job (Code Review + Refactor focus):

- Tests act as living documentation — you can refactor safely because tests catch regressions.
- Forces you to think about the interface before the implementation.

中文解释：

TDD = 先写测试，再写实现，最后重构。循环是：红（测试失败）→ 绿（测试通过）→ 重构（改善代码质量）。

对这个JD的意义：岗位要求Code Review和Code Refactor，TDD提供了重构的安全网——有测试覆盖才能放心重构，改坏了测试立刻报红。

关键点：每次只写刚好能让测试通过的最少代码，不过度设计，然后再重构。这叫做YAGNI原则（You Aren't Gonna Need It）。

Q7. How do you use Mockito in unit tests? Why do we mock dependencies?

问题7：如何在单元测试中使用 Mockito？为什么要 Mock 依赖？

English Answer:

When we test a service, we don't want to connect to a real database or call a real external API. That would make tests slow and unreliable. **Mockito** lets us create fake ("mock") versions of dependencies.

Example — test portfolio-service without hitting the real database:

```
@ExtendWith(MockitoExtension.class)
class PortfolioServiceTest {

    @Mock
    private PortfolioRepository repository; // fake database

    @Mock
    private FundClient fundClient; // fake HTTP call to fund-service

    @InjectMocks
    private PortfolioService portfolioService;

    @Test
    void shouldReturnTotalValueOfPortfolio() {
        // Arrange – tell the mock what to return
        when(repository.findById("user123"))
            .thenReturn(List.of(new Holding("FUND_A", 100)));
        when(fundClient.getPrice("FUND_A"))
            .thenReturn(new BigDecimal("12.50"));

        // Act
        Money total = portfolioService.getTotalValue("user123");

        // Assert
        assertThat(total.getAmount()).isEqualToComparingTo("1250.00");

        // Verify the repository was called exactly once
        verify(repository, times(1)).findById("user123");
    }
}
```

Why mock?

- Tests run in milliseconds, not seconds
- No need for a running database in CI pipeline

- You can test error cases easily: `when(...).thenThrow(new RuntimeException("DB down"))`

中文解释：

Mockito核心用法三步：

1. `@Mock`：创建假对象，不需要真实的数据库或网络连接。
2. `when(...).thenReturn(...)` / `when(...).thenThrow(...)`：定义假对象的行为。
3. `verify(...)`：验证某个方法被调用了几次。

为什么要Mock：单元测试要快（毫秒级）、稳定（不依赖外部系统）、可控（能模拟各种异常场景）。Mock让我们把测试范围严格限制在当前Service的业务逻辑上，而不是整个系统。

Q8. What is the Pub/Sub pattern? How does it help in a microservices system?

问题8：什么是 Pub/Sub 模式？它如何帮助微服务解耦？

English Answer:

Pub/Sub (Publish/Subscribe) is an Enterprise Integration Pattern. It works like a newspaper:

- The **Publisher** writes an article and sends it to the newspaper.
- Multiple **Subscribers** read the newspaper independently. The publisher does not know who reads it.

In microservices with Kafka:

```
order-service publishes: OrderPlaced { orderId, userId, amount, timestamp }  
    ↓ Kafka topic: "order.events"  
    ├── portfolio-service subscribes → updates user's holdings  
    ├── audit-service subscribes → writes to audit log  
    └── notification-service subscribes → sends email to user
```

Benefits:

1. **Loose coupling** — `order-service` does not call `portfolio-service` directly. If `portfolio-service` is down, orders still work. Portfolio-service catches up when it restarts.

2. **Easy to add new subscribers** — if we add a `fraud-detection-service`, it just subscribes to the same topic. No changes to `order-service`.
3. **Event replay** — Kafka keeps events for 7 days. If `portfolio-service` has a bug and processes events wrong, we can fix the bug and replay all events to correct the data.

Compare to direct REST calls: If `order-service` calls `portfolio-service` via REST and `portfolio-service` is down, the order fails. With Pub/Sub, the order succeeds and `portfolio-service` processes the event when it comes back up.

中文解释：

Pub/Sub是Enterprise Integration Patterns（企业集成模式）中的核心模式。发布者不知道谁在订阅，订阅者不知道发布者的实现细节，两者通过消息通道完全解耦。

在Kafka中的实践：`order-service` 发布 `OrderPlaced` 事件到Kafka Topic, `portfolio-service`、`audit-service`、`notification-service` 各自独立消费，互不影响。

核心价值：

1. **故障隔离**：`portfolio-service`宕机不影响下单，重启后从Kafka消费未处理的事件即可"追平"。
2. **易扩展**：新增订阅者只需订阅Topic，无需修改发布者代码（开闭原则）。
3. **事件回放**：Kafka保留历史消息，可以重新消费历史事件修复数据问题。

Q9. How do you use Redis in a microservices system? What problems does it solve?

问题9：在微服务系统中如何使用 Redis？它解决了什么问题？

English Answer:

We use Redis in several ways:

1. **Session cache / JWT token store** After a user logs in, we store their session in Redis with a TTL (e.g., 30 minutes). All microservices can check Redis to validate the token — no need to call the user-service every time.
2. **Database query cache** Fund information (prices, names) does not change every second. We cache it in Redis for 60 seconds. This reduces the load on Aurora by 80-90% for read-heavy endpoints.

```
@Cacheable(value = "fund-price", key = "#fundCode")
public FundPrice getFundPrice(String fundCode) {
    return fundRepository.findByCode(fundCode); // only called if not in
    cache
}
```

3. **Distributed lock** If two pods try to update the same user's portfolio at the same time, we can use a Redis lock (Redisson) to make sure only one runs at a time.
4. **Rate limiting** Use Redis `INCR` + `EXPIRE` to count API calls per user per minute. Block users who send too many requests.

中文解释：

Redis在微服务中的4种典型用途：

1. **会话缓存**：用户登录后把Session/JWT存Redis（带TTL），所有服务验证token时查Redis，避免每次都请求user-service。
2. **查询缓存**：基金净值等读多写少的数据缓存到Redis（60秒TTL），Spring Boot用 `@Cacheable` 注解一行搞定，大幅降低数据库压力。
3. **分布式锁**：多Pod并发修改同一用户数据时，用Redisson的分布式锁保证串行执行，防止数据竞争。
4. **限流计数器**：用 `INCR` + `EXPIRE` 实现API频率限制，防止接口被滥用。

Q10. What do you look for when doing a code review?

问题10：做 Code Review 时，你重点关注哪些方面？

English Answer:

I look at five areas when reviewing code:

1. **Correctness — does it work?**
 - Does the logic handle edge cases? (null input, empty list, negative numbers)
 - Are there off-by-one errors in loops?
 - Is the transaction boundary correct? (Does `@Transactional` cover the right methods?)
2. **Security — is it safe?**

- Any SQL injection risk? (Is the developer using `JdbcTemplate` with `?` placeholders, not string concatenation?)
- Is sensitive data (passwords, card numbers) being logged?
- Are authorization checks in the right place?

3. Design — is it clean?

- Does the method do only one thing? (Single Responsibility)
- Are variable names clear? (`userList` not `list1`)
- Is there duplicated code that should be extracted to a shared method?

4. Test coverage — is it tested?

- Is there a unit test for the new logic?
- Does the test cover the happy path and at least one error path?

5. Performance — will it scale?

- Is there an N+1 query problem? (Calling the database inside a loop)
- Are database indexes used correctly?

中文解释：

Code Review五个维度：

1. **正确性**：边界条件（null、空集合）、事务边界（`@Transactional` 范围是否正确）、并发安全。
2. **安全性**：SQL注入（要用参数化查询）、敏感信息不能进日志、权限校验位置是否正确。
3. **设计质量**：单一职责（一个方法只做一件事）、命名是否清晰、有无重复代码需要抽取。
4. **测试覆盖**：新逻辑有没有单元测试，有没有覆盖异常分支。
5. **性能**：有没有N+1查询（在循环里查数据库）、分页是否正确（避免一次加载全量数据）。

回答这道题时，可以提一个自己发现过的真实Bug，面试官印象会很深刻。

Q11. What is the Circuit Breaker pattern? Why is it important in a microservices system?

问题11：什么是断路器模式？它在微服务中为什么重要？

English Answer:

In a microservices system, services call each other. If `portfolio-service` calls `fund-service` and `fund-service` is slow or down, `portfolio-service` will keep waiting. Soon all its threads are stuck waiting. Then `portfolio-service` becomes slow too. This problem spreads through the whole system — it is called a **cascading failure**.

Circuit Breaker stops this from happening. It works like an electrical circuit breaker in your house:

- **Closed (normal)**: requests pass through normally.
- **Open (tripped)**: after 5 failures in a row, the circuit breaker "trips." All requests **immediately fail fast** with a fallback response (e.g., return cached data or an error message). No more waiting.
- **Half-open (testing)**: after 30 seconds, one test request is allowed through. If it succeeds, the circuit closes again.

```
// Spring Boot 3 + Resilience4j
@CircuitBreaker(name = "fundService", fallbackMethod = "getFundPriceFallback")
public FundPrice getFundPrice(String fundCode) {
    return fundClient.getPrice(fundCode); // external call
}

public FundPrice getFundPriceFallback(String fundCode, Exception ex) {
    return cache.getLastKnownPrice(fundCode); // return stale cache instead of
    failing
}
```

中文解释：

微服务级联故障的场景：A服务调B服务，B服务变慢，A的线程全部阻塞在等待B，A也变慢，C依赖A也开始变慢——整个系统雪崩。

断路器（Circuit Breaker）是防雪崩的核心手段：

- **Closed（关闭/正常）**：请求正常通过。
- **Open（打开/断路）**：连续失败N次后断路，后续请求直接走fallback（返回缓存数据或友好错误），不再等待，防止资源耗尽。
- **Half-Open（半开/探测）**：断路后等待一段时间，放一个探测请求，成功则恢复正常，失败继续断路。

在Spring Boot 3中通常用**Resilience4j**实现（Hystrix已停止维护）。

Part 2 — High Availability & Disaster Recovery

第二部分：高可用 / 两岸三中心

Q12. Can you explain what "two regions, three centers" means in your system?

问题12：请解释你们系统的"两岸三中心"设计是什么？

English Answer:

"Two regions, three centers" means we put our system in two different places and have three data centers in total.

In our system:

- **Region 1 — Singapore** (main site, "生产中心 + 同城灾备"):
We run our EKS cluster and Aurora database across **3 Availability Zones (AZ-a, AZ-b, AZ-c)** inside Singapore. AZ-a is the main production center. AZ-b and AZ-c are the local backup centers. If one AZ goes down, traffic automatically moves to another AZ in the same city. This is called **active-active within the same region**.
- **Region 2 — Hong Kong** (remote DR site, "异地灾备中心"):
We use **Aurora Global Database** to copy all data from Singapore to Hong Kong in real time. The delay is less than 1 second. If the whole Singapore region fails, we can switch to Hong Kong in under 15 minutes.

So the three centers are:

1. **Production Center** — Singapore, primary AZ
2. **Local DR Center** — Singapore, secondary AZs (same city, different buildings)
3. **Remote DR Center** — Hong Kong (different city, different country)

中文解释：

"两岸三中心"是金融行业的高可用标准：

- **两岸** = 两个地区：新加坡（主站） + 香港（异地灾备）
- **三中心** = 三个数据中心：
 1. 生产中心：新加坡，EKS主集群 + Aurora Writer

- 2. 同城灾备：新加坡另一个可用区，Aurora Reader副本
- 3. 异地灾备：香港，Aurora Global Database 只读副本（延迟 < 1秒）

当新加坡整个Region故障时，香港可以在15分钟内接管所有流量（手动升级Aurora副本为主库）。

Q13. What is RTO and RPO? What are your targets?

问题13：什么是 RTO 和 RPO？你们的目标是什么？

English Answer:

- **RTO (Recovery Time Objective)** = "How fast can we restart?" — the maximum time we allow the system to be down.
- **RPO (Recovery Point Objective)** = "How much data can we lose?" — the maximum data loss we can accept.

Failure Type	RTO	RPO	How We Do It
One Pod crashes	< 30 sec	0	Kubernetes restarts it automatically
One Node fails	< 2 min	0	Cluster Autoscaler adds a new Node
One AZ goes down	< 5 min	0	Pods are spread across 3 AZs; Aurora has Multi-AZ
Singapore region fails	< 15 min	< 5 min	Promote Hong Kong Aurora to primary; redeploy EKS
Someone deletes data by mistake	< 30 min	< 5 min	Aurora PITR (Point-in-Time Recovery)

中文解释：

- **RTO（恢复时间目标）**：系统宕机后，最多允许多长时间恢复服务。
- **RPO（恢复点目标）**：最多允许丢失多长时间的数据。

关键设计：Aurora Global Database 的跨区复制延迟 < 1秒，所以香港的数据几乎是实时的，RPO < 5分钟。

Q14. How does your system stay available when one Availability Zone (AZ) goes down?

问题14：当一个可用区（AZ）宕机时，系统如何保持可用？

English Answer:

We use three key tools to handle AZ failures:

1. **Pods are spread across all AZs** — we use `topologySpreadConstraints` in Kubernetes. This forces the system to put pods in different AZs. For example, if we have 6 pods, we put 2 in AZ-a, 2 in AZ-b, and 2 in AZ-c. If AZ-a goes down, the other 4 pods in AZ-b and AZ-c still work.
2. **Aurora Multi-AZ** — Aurora automatically keeps a copy of the database in each AZ. If the writer in AZ-a fails, Aurora takes about 30 seconds to promote a reader in AZ-b to become the new writer. The application does not need to change any code.
3. **RDS Proxy** — This sits between our application and Aurora. When Aurora fails over, RDS Proxy automatically connects to the new writer. The application just sees a short pause, then continues working.

中文解释：

三道保障：

1. **Kubernetes topologySpreadConstraints**：强制Pod跨AZ均匀分布，一个AZ宕机，剩余Pod继续处理请求。
2. **Aurora Multi-AZ**：数据库自动在多个AZ有副本，主库故障后30秒内自动切换到备库。
3. **RDS Proxy**：连接池代理，感知Aurora故障切换，应用层无需修改连接串，透明切换。

Q15. If Singapore is completely down, how do you switch to Hong Kong?

问题15：如果新加坡整个Region宕机，如何切换到香港？

English Answer:

This is our **cross-region disaster recovery** plan. Here are the steps:

Step 1 — Promote the Hong Kong database (< 1 minute)

We run one AWS command to make the Hong Kong Aurora cluster the new primary (writer):

```
aws rds failover-global-cluster \  
  --global-cluster-identifier smart-invest-global \  
  --target-db-cluster-identifier arn:aws:rds:ap-east-1:...:cluster:smart-  
invest-dr
```

Step 2 — Redirect traffic (< 5 minutes)

We update Route 53 DNS to point `app.smartinvest.com` to the Hong Kong load balancer. Because we use low TTL (60 seconds), DNS updates propagate quickly.

Step 3 — Start EKS workloads in Hong Kong (< 10 minutes)

We use ArgoCD and Kustomize. The Hong Kong EKS cluster already has all the Kubernetes configs stored in Git. We just apply them.

Total time: under 15 minutes.

We run this drill every quarter to make sure it works.

中文解释：

跨Region切换3步走：

1. 提升香港Aurora为主库：一条AWS CLI命令，约1分钟完成。
2. 切换Route 53 DNS：将域名指向香港的ALB，低TTL保证快速生效。
3. 香港EKS启动工作负载：ArgoCD从Git拉取配置，自动部署所有微服务。

全程不超过15分钟。我们每季度进行一次演练，确保流程有效。

Q16. How does Amazon Aurora's internal architecture work across multiple AZs? How is it different from traditional databases?

问题16：Amazon Aurora 在多可用区架构中内部是如何工作的？和传统数据库有何不同？

English Answer:

This is one of the most commonly misunderstood parts of Aurora. The key difference is that Aurora **separates storage from compute**, while traditional databases keep them together.

Traditional Multi-AZ (e.g., RDS MySQL):

- Each AZ has a completely independent, full copy of the database.

- Replication happens by shipping transaction logs from the primary to the standby.
- If the primary fails, the standby takes over — but you may lose a few seconds of data (RPO > 0).

Amazon Aurora Multi-AZ:

Storage layer (shared across all AZs):

- Aurora has a single distributed **Cluster Volume** that spans all 3 AZs.
- Every write is automatically stored in **6 copies across 3 AZs** (2 copies per AZ).
- A write is confirmed as soon as **4 of the 6** copies acknowledge it (quorum write). This means the system survives losing an entire AZ without losing any committed data.

Compute layer (separate roles):

- **AZ-a — Primary Instance (Writer):** handles all write operations and can serve reads.
- **AZ-b and AZ-c — Aurora Replicas (Readers):** handle read traffic, lag behind the writer by only a few milliseconds.
- If the writer crashes, Aurora promotes a reader to writer in **~30 seconds** — no data synchronisation needed because all nodes already share the same storage.

This design gives Aurora two key advantages over traditional Multi-AZ: **faster failover** (30 seconds vs several minutes) and **built-in read scaling** without manual replication setup.

中文解释:

Aurora 与传统数据库的核心区别在于"存储与计算分离":

传统 **Multi-AZ** (如 **RDS MySQL**) :

- 每个AZ各有一份完整、独立的数据库, 通过复制事务日志进行同步
- 主库宕机时, 备库接管, 但可能丢失几秒数据 (RPO > 0)

Amazon Aurora Multi-AZ:

存储层 (跨AZ共享) :

- Aurora 底层是一个跨3个AZ的共享集群卷 (Cluster Volume)
- 每次写入自动在3个AZ存储6份 (每个AZ 2份), 只需4/6副本确认即算成功 (Quorum写入)
- 即使整个AZ宕机, 已提交的数据不会丢失 (RPO = 0 for AZ failure)

计算层 (角色分工) :

- **AZ-a: Primary Instance (写节点)** — 处理所有写操作

- **AZ-b/AZ-c: Aurora Replica (读副本)** — 分担读流量，延迟仅几毫秒
- 主节点故障后，约30秒内自动提升读副本为新主节点，无需等待数据同步（存储本来就是共享的）

面试关键一句话: "**Aurora 的存储层是跨AZ的分布式共享卷，计算层才有 Writer 和 Reader 的区分，这与传统数据库每个AZ维护一份完整独立拷贝的模式根本不同。**"

Q17. What is the difference between hot standby, warm standby, and cold standby? Which does your Hong Kong DR site use and why?

问题17: 热备、温备、冷备有什么区别? 你们香港灾备中心用的是哪种模式, 为什么?

English Answer:

These terms describe how ready your backup site is to take over when a disaster happens:

Mode	Description	Recovery Time	Cost
Cold Standby	No resources running. Infrastructure is defined in Terraform/CDK but not deployed. Everything must be provisioned from scratch at disaster time.	Hours	Lowest
Warm Standby	Infrastructure exists (EKS cluster, load balancer, Aurora replica), but scaled down to a minimum. Scaled up at disaster time.	10–30 minutes	Medium
Hot Standby (Active-Active)	Full production capacity running in both sites simultaneously. Live traffic already flows to both.	Seconds	Highest (~2× cost)

Our Hong Kong DR site uses Warm Standby:

- The EKS cluster already exists in Hong Kong but runs only 2–3 nodes (production Singapore runs 30+).
- Aurora Global Database continuously replicates data from Singapore to Hong Kong with < 1 second lag — so the data is always current.
- When disaster strikes, Karpenter provisions new EKS nodes in ~2 minutes, we promote Aurora to primary in ~1 minute, and ArgoCD deploys all services in ~5 minutes.

- **Total: ~15 minutes → meets our RTO target.**

We chose Warm Standby over Hot Standby to control cost. Running full production capacity in two regions simultaneously would nearly double our infrastructure spend, which is hard to justify at our current scale.

中文解释：

三种灾备模式的核心区别是"平时烧多少钱，出事后多快能恢复"：

模式	平时状态	恢复时间	成本
冷备 (Cold Standby)	什么都没运行，只有Terraform配置	数小时	最低
温备 (Warm Standby)	基础设施存在但缩容至最低	10–30分钟	中等
热备/双活 (Hot Standby)	两边都跑满，同时承接真实流量	秒级	最高（约2倍成本）

我们香港用温备（Warm Standby）：

- EKS集群已存在，但只保留2–3个节点（新加坡生产环境有30+节点）
- Aurora Global Database 实时复制，香港数据延迟 < 1秒，时刻保持最新
- 灾难触发后：Karpenter 约2分钟扩容EKS节点，提升 Aurora 主库约1分钟，ArgoCD 部署微服务约5分钟
- 合计约15分钟，满足 **RTO < 15分钟**目标

选择温备而非热备的原因：热备需要香港维持与新加坡同等规模的基础设施，成本接近翻倍。在我们当前的业务规模下，这笔额外支出不经济。

Q18. During the 15-minute failover to Hong Kong, what happens to in-flight requests that were being processed in Singapore?

问题18：在15分钟的切换窗口内，正在新加坡处理中的请求（in-flight requests）会怎么样？

English Answer:

This is a realistic question about what users actually experience during a regional failover. The honest answer is: **some requests will fail**, and the system must be designed to handle this

gracefully.

Here is what happens during the ~15-minute window:

1. **Active connections to Singapore are dropped.** Any request being processed in Singapore will time out or return an error. Users will see an error page or experience a service interruption.
2. **The application handles this through three mechanisms:**
 - **Retry logic for idempotent operations** — fetching a portfolio balance or market price can be retried safely without causing side effects. Clients retry automatically after a short backoff.
 - **Non-idempotent writes need extra care** — if a buy order was half-processed when Singapore went down, we must verify on recovery whether it was committed or not. We use Kafka's exactly-once semantics and database transaction boundaries to ensure a trade is never double-executed.
 - **Circuit Breaker (Resilience4j)** — detects repeated failures to the Singapore endpoint and stops sending new requests there, preventing cascading failures across services.
3. **Route 53 health checks detect the outage** — Route 53 polls the Singapore ALB every 10 seconds. When it detects failure, it automatically updates DNS to point new requests to the Hong Kong load balancer. Because our DNS TTL is 60 seconds, most clients see Hong Kong within 1–2 minutes of the switchover.
4. **What users actually experience:** a service interruption of roughly 15 minutes. For a financial application, we communicate this through status page updates and push notifications.

The key design principle: every write operation must be idempotent — safely re-executable without producing double charges, duplicate orders, or corrupted account balances.

中文解释：

这道题考察你是否理解灾难恢复的真实代价。诚实的回答是：**在15分钟切换窗口内，部分请求会失败，系统必须在设计层面应对这一事实。**

具体发生什么：

1. **新加坡的活跃连接被强制中断** — 正在处理的请求超时或报错，用户看到错误页面。
2. **应用层通过三种机制应对：**
 - **幂等操作自动重试** — 查询持仓、查询余额等操作可以安全重试，不影响正确性

- **非幂等写操作需要特别保障** — 例如买入订单在新加坡处理了一半就断开了，切换到香港后需要确认该订单是否已提交。我们通过 Kafka Exactly-Once + 数据库事务来确保不会重复扣款
- **Circuit Breaker (Resilience4j 熔断器)** — 检测到新加坡端点连续失败后自动熔断，阻止新请求继续打到宕机的端点，防止服务雪崩

3. **Route 53 健康检查自动触发** — Route 53 每10秒轮询新加坡ALB，检测到故障后自动将DNS切向香港ALB。因为我们设置了60秒TTL，1–2分钟内大部分客户端就能访问香港。

4. **用户实际体验** — 约15分钟的服务中断窗口。我们通过状态页面和推送通知告知用户。

核心设计原则：所有写操作必须是幂等的 (**Idempotent**)，才能在灾难恢复后安全重试，不产生重复扣款、重复下单等金融事故。

Q19. How does Aurora Global Database prevent split-brain during a cross-region failover?

问题19: Aurora Global Database 在跨Region切换时，如何防止脑裂 (Split-Brain) 问题?

English Answer:

Split-brain means two database instances both believe they are the primary writer at the same time. In a financial system, this is catastrophic — you could end up with two conflicting records of the same trade.

Aurora Global Database prevents split-brain through a **managed, exclusive promotion process**:

1. **Only one region can write at any time.** The Aurora Global Database architecture enforces a single writer at the storage layer — not just at the application layer. All other clusters are strictly read-only replicas.
2. **The promotion operation is atomic.** When you run `aws rds failover-global-cluster`, AWS performs these steps atomically:
 - Detaches the Hong Kong cluster from the global cluster, making it an independent primary (read-write).
 - Simultaneously marks Singapore as "no longer primary" — it cannot accept writes even if it recovers.
 - There is no intermediate state where both clusters are writable.

3. **Singapore cannot write once detached.** Even if Singapore comes back online, it is now an isolated, detached cluster. It no longer belongs to the global cluster and cannot produce writes that affect Hong Kong's data.
4. **Re-joining after recovery.** After the incident, Singapore re-joins the global cluster as a **read replica** (not as primary). You verify data consistency before promoting it back to primary.

The critical insight: Aurora prevents split-brain **at the infrastructure level**, not through application-layer heartbeats or lock files. You simply cannot have two writers in the same Aurora Global Database cluster simultaneously.

中文解释：

脑裂 (Split-Brain) = 两个数据库实例同时认为自己是主库，各自接受写入，导致同一笔交易产生两份不一致的记录。在金融系统中，后果是灾难性的。

Aurora Global Database 通过**托管式排他提升 (Managed Exclusive Promotion)**防止脑裂：

1. 同一时刻只允许一个**Region**可写 — Aurora Global 在**存储层**强制执行，只有一个Primary Cluster拥有写权限，不是靠应用层逻辑控制的。
2. 提升操作是原子的 — 执行 `failover-global-cluster` 时，AWS 原子性地完成：
 - 将香港集群从全球集群中"分离 (Detach)"，提升为独立的读写主集群
 - 同时将新加坡标记为"不再是Primary"，不能接受写操作
 - 不存在两个集群同时可写的中间状态
3. 新加坡即使恢复也无法写入 — 它已被从Global集群中移除，成为一个孤立集群，不会对香港的数据产生任何写入影响。
4. 灾后重建 — 确认香港数据完整后，将新加坡以**只读副本**身份重新加入全球集群，而非直接提升为主库。

关键点：Aurora Global Database 在**基础设施层**防止了脑裂，而非依赖应用层心跳或分布式锁等机制。一个全球集群中，同一时刻永远只能有一个可写节点。

Q20. How do you test your disaster recovery plan? Walk me through a DR drill.

问题20：如何测试灾备方案？走一遍 DR 演练 (Disaster Recovery Drill) 的完整流程。

English Answer:

A DR plan that has never been tested is just a theory. We run a **quarterly DR drill** to verify the entire failover process works end-to-end.

Phase 1 — Pre-drill preparation (1 week before)

- Notify all engineering teams and stakeholders. Schedule a maintenance window.
- Review and update the runbook (step-by-step operational instructions).
- Verify that Hong Kong EKS cluster and Aurora replica are healthy and in sync.
- Confirm that all Terraform and Kubernetes configs in Git are up to date.

Phase 2 — Simulate the failure

- Block all traffic to the Singapore ALB using Route 53 (we do NOT destroy Singapore — we need to be able to roll back quickly).
- Monitor CloudWatch alarms and PagerDuty alerts to confirm they fire as expected within the agreed SLA.

Phase 3 — Execute the failover (target: < 15 minutes)

1. Promote Hong Kong Aurora to primary — one AWS CLI command, ~1 minute.
2. Update Route 53 health check policy to route to Hong Kong ALB — ~2–3 minutes.
3. Karpenter scales up Hong Kong EKS nodes — ~2–3 minutes.
4. ArgoCD pulls configs from Git and deploys all microservices — ~5–7 minutes.
5. Run smoke tests: login, portfolio query, place a test order, check trade history.

Phase 4 — Measure and record

- Actual RTO achieved (target < 15 minutes).
- Actual RPO: check the timestamp of the last committed Aurora transaction in Hong Kong vs Singapore.
- Document every step that took longer than expected or required manual intervention.

Phase 5 — Fail back

- Re-add Singapore as a read replica of Hong Kong.
- Gradually restore traffic to Singapore.
- Write a post-drill report with lessons learned and action items.

What we learned from past drills: in one drill, we discovered that Karpenter in Hong Kong was not pre-configured with the correct node pool, adding 5 extra minutes. We fixed this before the

next drill. This is exactly the value of drilling — you cannot afford to discover configuration gaps during a real disaster.

中文解释：

没有测试过的灾备方案只是一张纸。我们**每季度**执行一次完整的灾备演练（DR Drill）。

第一阶段：提前准备（演练前一周）

- 通知所有工程团队和相关方，安排维护窗口
- 审查并更新 Runbook（逐步操作手册）
- 确认香港EKS集群和Aurora副本健康且数据同步正常
- 确认Git中所有Terraform和Kubernetes配置均为最新

第二阶段：模拟故障

- 通过 Route 53 屏蔽新加坡ALB的流量（不真正摧毁新加坡环境，便于快速回切）
- 监控 CloudWatch 告警和 PagerDuty，确认告警按预期在规定时间内触发

第三阶段：执行切换（目标：< 15分钟）

1. 提升香港 Aurora 为主库（一条 AWS CLI 命令，约1分钟）
2. 更新 Route 53 将流量路由至香港 ALB（约2–3分钟）
3. Karpenter 扩容香港 EKS 节点（约2–3分钟）
4. ArgoCD 从 Git 拉取配置，部署所有微服务（约5–7分钟）
5. 执行冒烟测试：登录、查询持仓、下测试订单、查看交易记录

第四阶段：测量与记录

- 记录实际达成的 RTO（目标 < 15分钟）
- 记录实际 RPO：对比香港与新加坡 Aurora 最后一笔已提交事务的时间戳
- 记录所有超时或需要手动干预的步骤

第五阶段：回切

- 将新加坡以只读副本身份重新加入全球集群
- 逐步将流量切回新加坡
- 编写演练报告，记录改进项和行动事项

真实经验：某次演练中我们发现香港的Karpenter节点池配置未同步，导致扩容多花了5分钟。在下次演练前完成了修复。演练的价值正在于此——绝不能等到真正的灾难才发现配置漏洞。

Q21. Why do you put NAT Gateways in public subnets, but EKS and Aurora in private subnets?

问题21：为什么 NAT Gateway 放在公有子网，而 EKS 和 Aurora 放在私有子网？

English Answer:

This is a security design called "**defense in depth**" — we add many layers of protection.

- **Private subnets** have no direct connection to the internet. Attackers cannot reach our EKS pods or Aurora database directly.
- **EKS pods** need to call external APIs (like sending emails). They go through the **NAT Gateway** in the public subnet. NAT Gateway lets them call the internet, but blocks anyone from the internet calling them back.
- **Aurora** never needs to talk to the internet at all. It only talks to EKS pods inside the VPC. So we put it in the most private subnet.

We also use **VPC Endpoints** so that EKS pods can talk to AWS services (like Secrets Manager, ECR, CloudWatch) without the traffic leaving the AWS network at all.

中文解释：

这是"纵深防御"网络设计：

- **公有子网**：只放 NAT Gateway 和 ALB，暴露最小的攻击面。
- **私有子网（App层）**：EKS节点，外部无法直接访问，出向流量通过NAT Gateway。
- **私有子网（Data层）**：Aurora和Redis，完全不对外，只允许来自EKS的流量（Security Group白名单）。
- **VPC Endpoint**：访问Secrets Manager、ECR等AWS服务走AWS内网，不经过公网，进一步减少攻击面。

Part 3 — EKS & Microservices

第三部分：EKS 与微服务

Q22. Why did you choose EKS instead of ECS Fargate?

问题22：为什么选择 EKS 而不是 ECS Fargate?

English Answer:

Both EKS and ECS can run containers. We chose EKS for three main reasons:

1. **Network Policy** — EKS lets us use Calico or AWS VPC CNI to set rules like "service A cannot talk to service B." ECS Fargate does not have this feature. For a financial system, we need to strictly control which service can talk to which database.
2. **PodDisruptionBudget (PDB)** — In EKS, we can say "always keep at least 1 portfolio-service pod running." This protects us during maintenance or node upgrades. ECS does not have a direct equivalent.
3. **More control** — EKS is standard Kubernetes. We can use any open-source Kubernetes tool (Prometheus, cert-manager, Karpenter, ArgoCD). ECS is AWS-specific and has fewer options.

The trade-off is that EKS is more complex to manage. But for a financial-grade system, the extra control is worth it.

中文解释:

选择EKS的3个关键原因:

1. **Network Policy (网络隔离)**: EKS + Calico可以实现细粒度的Pod间网络访问控制, 这是金融合规要求的。ECS Fargate不支持。
2. **PodDisruptionBudget**: 维护期间保证最少可用Pod数量, ECS没有等价功能。
3. **生态系统**: 标准Kubernetes, 可以使用所有开源工具 (Prometheus、ArgoCD、Karpenter等)。

Q23. What is IRSA and why is it better than putting AWS keys in the code?

问题23：什么是 IRSA？为什么比把 AWS 密钥放在代码里更好？

English Answer:

IRSA stands for **IAM Roles for Service Accounts**. It is a way to give each Kubernetes pod permission to use AWS services, without using any password or access key.

Old way (bad): Put `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` as environment variables in the pod. Problems:

- If someone reads the pod config, they can steal the key.
- The key never expires automatically.
- All pods share the same key, so one compromised pod puts everything at risk.

IRSA (good):

- Each pod gets its own IAM Role. For example, `portfolio-service` gets a role that can only read its own secrets from Secrets Manager.
- There is no password to steal. AWS gives the pod a short-lived token (expires every hour).
- Tokens rotate automatically.

This follows the **principle of least privilege** — each service can only do exactly what it needs, nothing more.

中文解释:

IRSA (IAM Roles for Service Accounts) 解决了"如何让Pod安全访问AWS资源"的问题:

- **旧方式 (危险)** : 把AK/SK硬编码或放入环境变量, 密钥长期有效, 一旦泄露后果严重。
- **IRSA (安全)** : Pod通过K8s ServiceAccount绑定IAM Role, 获取自动轮换的临时Token (每小时过期), 无需任何长期密钥。每个服务只有访问自己资源的权限 (最小权限原则) 。

Q24. How does the system handle a sudden traffic spike, like 10x more users?

问题24: 如果流量突然增加10倍, 系统如何应对?

English Answer:

We use a **two-level autoscaling** system:

Level 1 — Pod scaling (HPA):

Kubernetes HPA (Horizontal Pod Autoscaler) watches CPU and memory usage. When CPU goes above 60%, it adds more pods. For example, `portfolio-service` can scale from 2 pods to 20 pods automatically in about 15 seconds.

Level 2 — Node scaling (Karpenter):

When there are more pods than the current nodes can hold, Karpenter automatically adds new

EC2 instances to the cluster. Karpenter is faster than the old Cluster Autoscaler — it can add a new node in about 60 seconds.

Scale-down protection:

When traffic drops, we wait 5 minutes (`stabilizationWindowSeconds: 300`) before removing pods. This avoids "flapping" — removing pods too quickly and then needing to add them back.

中文解释：

两级弹性伸缩：

1. **Pod级（HPA）**：CPU超过60%或内存超过70%时，自动增加Pod数量（最多20个）。扩容无等待，缩容有5分钟稳定窗口防止抖动。
2. **Node级（Karpenter）**：当Node资源不足时，Karpenter自动添加新的EC2实例，速度比传统Cluster Autoscaler快约2倍（约60秒）。

极端情况下，系统可以在2-3分钟内从最小配置扩展到最大配置，无需人工干预。

Part 4 — Security

第四部分：安全设计

Q25. How does the system protect against DDoS attacks?

问题25：系统如何防御 DDoS 攻击？

English Answer:

We use a **three-layer defense**:

Layer 1 — AWS Shield Standard (automatic, free)

This protects against large network attacks (like SYN floods and UDP floods) at the CloudFront level. It's always on and works 24/7.

Layer 2 — AWS WAF (Web Application Firewall)

WAF runs on CloudFront and checks every HTTP request. It blocks:

- SQL Injection (attacker puts SQL code in a form field)
- XSS (Cross-Site Scripting)

- Known bad bots
- Rate limiting: if one IP sends more than 1000 requests in 5 minutes, it gets blocked automatically.

Layer 3 — CloudFront absorbs the traffic

CloudFront is a global CDN. It handles attacks at the edge, before traffic even reaches our servers in Singapore or Hong Kong. This means our EKS cluster and Aurora database are never directly hit.

中文解释：

三层DDoS防护：

1. **Shield Standard**：自动防护L3/L4网络层攻击（流量清洗），免费，部署在CloudFront。
2. **WAF**：防护L7应用层攻击，规则包括SQL注入、XSS、恶意Bot，还有频率限制（1000次/5分钟/IP）。
3. **CloudFront边缘节点**：攻击流量在边缘被吸收和过滤，不会到达新加坡或香港的服务器。

Q26. How does the system store and protect passwords and secrets?

问题26：系统如何存储和保护密码、密钥等敏感信息？

English Answer:

We follow one rule: **no secret is ever stored in code, config files, or Kubernetes Secrets in plain text.**

Here's how it works:

1. **AWS Secrets Manager** stores all secrets:
 - Database password (auto-rotated every 30 days)
 - JWT signing key (rotated every 90 days)
 - Redis auth token
2. **AWS KMS (Key Management Service)** encrypts the secrets inside Secrets Manager. We use our own Customer Managed Keys (CMK), not AWS's default keys.
3. **Secrets Store CSI Driver** injects secrets into the pod as mounted files at runtime. The secret is never stored in a Kubernetes Secret object (which is only base64-encoded, not truly encrypted).

4. **Aurora** uses **IAM Database Authentication** — instead of a username/password, the pod uses its IAM token to connect. No password needed.

中文解释：

零明文密钥原则：

- **Secrets Manager**：集中存储所有密钥，数据库密码每30天自动轮换，无需重启应用（RDS Proxy会透明切换）。
 - **KMS（客户管理密钥）**：对Secrets Manager中的数据做信封加密，密钥控制权在我们手中。
 - **Secrets Store CSI Driver**：Pod启动时从Secrets Manager拉取密钥挂载为文件，不写入K8s Secret（K8s Secret只是base64，不是加密）。
 - **Aurora IAM认证**：Pod用IAM临时Token连接数据库，根本没有数据库密码。
-

Q27. Someone on your team accidentally pushes a bad Docker image to production. How does the system catch this?

问题27：如果团队成员不小心把有问题的 **Docker** 镜像推送到了生产环境，系统如何防御？

English Answer:

We have **multiple checkpoints** before any image reaches production:

1. **GitHub Actions CI** — every pull request runs:
 - Unit tests and integration tests
 - OWASP dependency scan (checks for known security vulnerabilities in libraries)
 - Amazon Inspector scans the Docker image for CVEs (known security holes)
 - If any check fails, the image cannot be pushed to ECR.
2. **Image Signing** — after the image passes all checks, we sign it with cosign (or AWS Signer). A Kyverno policy on EKS checks: "Is this image signed?" If not, the pod cannot start.
3. **ArgoCD + manual approval gate** — changes go through: dev → staging → production. The production step requires a human to approve.
4. **Canary rollout** — even after approval, we only send 10% of traffic to the new version first. We watch the error rate. If errors go above 1%, ArgoCD automatically stops the rollout and

rolls back.

中文解释：

4道安全闸门：

1. **CI门禁**：PR阶段运行单元测试、依赖扫描、Inspector镜像CVE扫描，全部通过才能推送到 ECR。
2. **镜像签名**：只有通过CI签名的镜像才能在EKS中运行（Kyverno Policy强制校验签名），防止供应链攻击。
3. **手动审批门**：生产环境部署需要人工确认（GitOps manual approval gate）。
4. **Canary金丝雀发布**：先放10%流量，监控错误率，超过1%自动回滚，保护99%的用户。

Part 5 — Database Design

第五部分：数据库设计

Q28. Why did you choose Aurora PostgreSQL instead of regular RDS PostgreSQL?

问题28：为什么选择 Aurora PostgreSQL 而不是普通的 RDS PostgreSQL?

English Answer:

Aurora PostgreSQL is better than regular RDS in three important ways for a financial system:

1. **Faster failover**: When the primary database fails, Aurora promotes a reader in about **30 seconds**. Regular RDS takes about **60–120 seconds**. In a financial system, 30 extra seconds of downtime can mean many failed transactions.
2. **Storage auto-expansion**: Aurora's storage grows automatically from 10GB up to 128TB. With regular RDS, you have to set a fixed size and manually resize.
3. **Global Database for DR**: Aurora Global Database can replicate data to Hong Kong with less than **1 second of delay**. This is how we achieve RPO < 5 minutes for cross-region disasters. Regular RDS does not have this feature.

The trade-off is that Aurora costs more than regular RDS. But for a financial product, reliability is more important than cost.

中文解释：

Aurora vs RDS三个关键差异：

1. **故障切换更快**：Aurora约30秒，RDS约60-120秒。金融业务每多一秒宕机损失都很大。
 2. **存储自动扩展**：Aurora从10GB自动扩展到128TB，无需手动操作，不影响业务。
 3. **Global Database**：跨Region复制延迟 < 1秒，这是我们能实现香港灾备RPO < 5分钟的技术基础，普通RDS没有这个功能。
-

Q29. What problem does RDS Proxy solve?

问题29：RDS Proxy 解决了什么问题？

English Answer:

Spring Boot applications create many database connections. In a microservices system with 50+ pods, each pod might keep 10 connections open. That's 500+ connections to Aurora, which is near Aurora's maximum limit.

RDS Proxy sits between the pods and Aurora:

- It **pools** connections. 500 pods can share a smaller number of actual database connections.
- When Aurora fails over (switches to a new primary), RDS Proxy **automatically reconnects** to the new primary. The application sees only a 1-2 second pause.
- When the database password rotates every 30 days, RDS Proxy fetches the new password from Secrets Manager automatically. We don't need to restart any pods.

中文解释：

RDS Proxy解决3个问题：

1. **连接数爆炸**：50个Pod × 每Pod10个连接 = 500个连接。Aurora的连接数有上限（约5000），RDS Proxy通过连接复用将实际到Aurora的连接控制在合理范围。
2. **故障切换透明化**：Aurora主备切换时，RDS Proxy自动感知新主库，应用层只感受到短暂停顿（1-2秒），无需修改连接串。
3. **密码轮换无感**：Secrets Manager每30天更换数据库密码，RDS Proxy自动拉取新密码，不需要重启Pod。

Part 6 — CI/CD & GitOps

第六部分：CI/CD 与 GitOps

Q30. Can you walk me through what happens when a developer merges code to main?

问题30：当开发者把代码合并到 **main** 分支时，发生了什么？请走一遍流程。

English Answer:

Here is the full journey from code to production:

```
Developer merges PR to main
```

```
↓
```

```
1. GitHub Actions starts (automatic)
```

- Runs unit tests (mvn test)
- Builds the JAR file
- Builds Docker image
- Amazon Inspector scans the image for CVEs
- Signs the image with cosign
- Pushes the image to ECR with a unique tag (git commit SHA)

```
↓
```

```
2. GitHub Actions updates the infra repo
```

- Changes the image tag in kustomization.yaml
- Commits and pushes to the infra Git repo

```
↓
```

```
3. ArgoCD detects the change (polls every 3 minutes or uses webhook)
```

- Deploys to staging automatically
- Runs smoke tests

```
↓
```

```
4. Human approves production deployment
```

- ArgoCD deploys to production
- Canary: 10% traffic → watch errors → 50% → 100%
- If error rate > 1%, ArgoCD rolls back automatically

The key idea is that **Git is the single source of truth**. If someone changes the cluster manually (e.g., with kubectl), ArgoCD detects the drift and resets it back to what Git says.

中文解释：

GitOps核心理念：**Git是唯一的事实来源。**

完整流程：代码合并 → CI（构建+测试+扫描+签名+推送ECR） → 更新infra仓库的镜像Tag → ArgoCD检测变更 → 自动部署到Staging → 人工审批 → Canary金丝雀发布到Production → 自动监控错误率，超阈值自动回滚。

任何人通过kubectl手动修改集群配置，ArgoCD都会检测到偏差（drift）并自动恢复到Git中的声明状态。

Q31. What is a "canary deployment"? Why do you use it instead of releasing to all users at once?

问题31：什么是"金丝雀发布"？为什么不直接发布给所有用户？

English Answer:

A canary deployment means you release a new version to only a **small percentage of users first** — say 10%. You watch how it behaves. If everything is fine, you slowly increase to 50%, then 100%. If something goes wrong, you only affected 10% of users, and you roll back immediately.

The name comes from an old practice where miners brought canary birds into coal mines. If the air was poisonous, the canary would fall sick first — a warning before humans were harmed.

In our system, ArgoCD Rollouts does this automatically:

- Send 10% of traffic to the new version
- Watch error rate for 5 minutes
- If error rate < 1%, continue to 50%
- If error rate > 1%, roll back to the old version automatically

Compare to a **big-bang deployment** (all users at once): if there's a bug, all users are affected, and rollback takes time. Canary deployment is much safer.

中文解释：

金丝雀发布：先把新版本暴露给少量用户（10%），观察是否有错误，再逐步扩大（50% → 100%）。

名字来源：早期矿工会带金丝雀进矿井，一氧化碳积累时金丝雀先昏倒，给矿工预警。新版本就是那只"金丝雀"——先承受可能的风险，保护大多数用户。

ArgoCD Rollouts自动执行：10% → 观察5分钟 → 错误率 < 1% → 继续扩大 → 否则自动回滚。
对比"全量发布"：一旦出bug影响100%用户，本方案最坏情况只影响10%用户。

Part 7 — Observability

第七部分：可观测性

Q32. If a user says "the system is slow," how do you find out what's wrong?

问题32：如果用户说"系统很慢"，你如何找到问题所在？

English Answer:

We use a **three-signal approach**: Metrics, Logs, and Traces.

Step 1 — Check Metrics (Grafana dashboard)

Open the RED dashboard (Rate / Error / Duration):

- Is the request rate (number of requests per second) normal?
- Is the error rate above 1%?
- Is the response time (P99 latency) high? For example, normally P99 = 200ms, but now it's 2000ms.

This tells us which service is slow.

Step 2 — Check Logs (CloudWatch Logs Insights)

Run a query to find error messages or slow requests in the specific service:

```
fields @timestamp, level, message, traceId
| filter level = "ERROR" or duration > 1000
| sort @timestamp desc
```

Step 3 — Check Traces (AWS X-Ray)

X-Ray shows the full journey of one request: `user-service → database → Redis`. We can see exactly which step is slow. For example: "The database query for this endpoint took 1800ms — that's the bottleneck."

中文解释：

排查"系统慢"三步法（Metrics → Logs → Traces）：

1. **Grafana RED看大盘**：Rate（请求量有没有异常峰值）、Error（错误率有没有升高）、Duration（P99延迟有没有变高），定位到哪个服务。
 2. **CloudWatch Logs查错误日志**：在问题服务中搜索ERROR日志或慢请求（duration > 1000ms），找到具体报错信息。
 3. **X-Ray看全链路Trace**：找到某一个慢请求，看完整调用链（HTTP → Service → RDS → Redis），精确定位是哪个环节耗时最长。
- 三者配合，通常5-10分钟内可以定位问题根因。

Quick Reference — Key Technology Choices

快速参考：核心技术选型理由

Technology	Why We Chose It	Why Not the Alternative
EKS	Network Policy, PDB, full K8s ecosystem	ECS Fargate: less control, no Network Policy
Aurora PostgreSQL	30s failover, Global DB, auto storage	Regular RDS: slower failover, no Global DB
RDS Proxy	Connection pooling, transparent failover	Direct connection: connection limit, slow failover
IRSA	No long-lived credentials, per-service least privilege	Hard-coded AK/SK: security risk, key rotation pain
ArgoCD GitOps	Git as source of truth, drift detection, audit trail	kubectl apply: no audit, no drift detection
Secrets Store CSI	Secrets never in K8s Secret (base64 only)	K8s Secret: base64 is not encryption
CloudFront + WAF	Edge protection, DDoS absorption, geo-restriction	Direct ALB exposure: no edge protection
Canary Deployment	Max 10% users affected if bug found	Big-bang: 100% users affected

面试小贴士：

- 每个技术选型都要说"为什么选它，为什么不选另一个"（trade-off）。
- 两岸三中心优先讲RTO/RPO数字，面试官印象深刻。
- 遇到不会的，说"In our current design we did X, but if I had more time I would also consider Y"。

Part 8 — Capacity & Numbers (QPS / Latency / Scale)

第八部分：容量设计题 — "你们系统的 QPS 是多少？"

这类题的本质：面试官想看你推算思路，而不是背一个数字。

答题公式：估算用户规模 → 推算并发 → 算出 QPS → 说明架构能否支撑 → 说出扩容方案

Q33. What is the QPS of your system? How did you design for it?

问题33：你们系统的 QPS 是多少？是怎么设计和支撑的？

English Answer:

I will walk you through how I estimated the QPS, then explain how the architecture handles it.

Step 1 — Estimate the user base

Flex Invest is a fund investment platform. It is not a high-frequency stock trading app. Users check their portfolio and browse funds — they do not click hundreds of times per second.

Metric	Estimate	Reasoning
Registered users	500,000	Medium-sized financial platform
Daily Active Users (DAU)	50,000	~10% of registered users log in each day
Peak concurrent users	5,000	~10% of DAU are online at the same time
Requests per user per minute	3	One click every 20 seconds on average

Step 2 — Calculate peak QPS

Peak QPS = concurrent users × requests per minute / 60 seconds
= 5,000 × 3 / 60
≈ 250 QPS (total across all services)

Breaking it down by service:

Service	QPS	Why
portfolio-service	~100 QPS	Most users check portfolio first
fund-service	~80 QPS	Browsing fund list and details
order-service	~30 QPS	People don't trade every second
user-service	~20 QPS	Login + profile update
plan-service	~20 QPS	Investment plan management

Peak spikes: Around market close (3pm Singapore time), order QPS can jump **5-10x** — up to 150-300 QPS just for order-service.

Step 3 — How the architecture handles it

Layer	Capacity	Our Load	Headroom
CloudFront (static)	Unlimited (CDN)	~60% of requests	✅ Infinite
ALB	~1M connections	5,000	✅ 200x margin
EKS (per service, 20 pods)	~2,000 QPS	~250 QPS peak	✅ 8x margin
Aurora (RDS Proxy)	~5,000 TPS	~30 TPS writes	✅ 166x margin
ElastiCache Redis	~100,000 ops/s	~500 ops/s	✅ 200x margin

Step 4 — If traffic grows 10x

HPA automatically scales each service from 2 pods to 20 pods. Karpenter adds new EC2 nodes in ~60 seconds. For sustained 10x load we would also upgrade Aurora reader instances and add Redis memory.

中文解释：

QPS估算公式（背下来）：

注册用户(500K) × DAU比例(10%) = 5万日活
5万日活 × 峰值在线比例(10%) = 5千并发
5千并发 × 每分钟3次请求 / 60秒 ≈ 250 QPS

下午3点收盘尖峰时 order-service 可达 300 QPS。EKS 20个Pod上限约2000 QPS，现有负载的8倍冗余。

关键话术: "We designed for 10x peak capacity because in financial systems, downtime during market close is not acceptable."

Q34. What is the P99 latency requirement? How do you measure it?

问题34：P99 延迟要求是多少？你怎么测量的？

English Answer:

P99 means: if we sort all requests by response time, P99 is the 99th percentile — 99% of requests finish faster than this number.

Our latency targets:

API Type	P50 (median)	P99 target	Why
Portfolio read (Redis cache hit)	< 20ms	< 100ms	Redis is in-memory
Portfolio read (Aurora query)	< 100ms	< 300ms	Database query
Fund browse (cached)	< 50ms	< 200ms	Redis cache
Order placement	< 200ms	< 500ms	Writes to Aurora + publishes Kafka event
User login	< 100ms	< 300ms	JWT generation + Redis session

How we measure:

1. **Prometheus + Grafana** — Spring Boot Actuator auto-records latency histograms. Grafana shows P50/P95/P99 per endpoint in real time.

- 2. **AWS X-Ray** — when P99 is high, X-Ray traces show exactly which step is slow (DB query? Redis? External API?).
- 3. **JMeter load tests** — before every release, we simulate 10x peak and verify P99 stays within target.

Alert: if P99 > 1 second on any endpoint, PagerDuty pages the on-call engineer.

中文解释：

P99 = 99%的请求在这个时间内完成，比平均值更有意义（平均值会被少数超快请求拉低）。

目标：读接口（Redis缓存）P99 < 100ms，写接口（下单）P99 < 500ms。监控工具：Prometheus采集延迟直方图，Grafana展示，X-Ray定位慢点，JMeter压测验证。P99 > 1秒触发PagerDuty告警。

Q35. How much data does your system store? How fast does it grow?

问题35：你们系统存了多少数据？增长速度是多少？

English Answer:

Data Type	Size	Monthly Growth
User profiles (500K users)	~3.5 GB	~10 MB
Portfolio snapshots (daily)	~5 GB	~200 MB
Order history	~10 GB	~500 MB
Fund data (prices, info)	~2 GB	~100 MB
Total Aurora data	~20 GB	~800 MB
Application logs	~5 GB/day	CloudWatch 90 days → S3 Glacier 7 years
Audit trail (CloudTrail)	~1 GB/day	S3 Object Lock, 7-year retention

Key points:

- 20 GB is tiny for Aurora (max 128 TB). No storage pressure.
- **Logs are the biggest cost driver.** We use S3 Lifecycle rules to move logs to Glacier after 90 days — cost drops 10x.
- 7-year audit retention is a **regulatory requirement** for financial systems, not optional.

中文解释：

Aurora 总数据量约 **20GB**，每月增长约 800MB，对 Aurora 微不足道（上限128TB）。最大存储消耗是日志（5GB/天），90天后归档到 S3 Glacier（成本降低10倍），满足金融监管7年合规保留要求。

Q36. Have you done load testing? How did you do it?

问题36：你们做过压力测试吗？怎么做的？

English Answer:

Yes. We run load tests before every major release using **JMeter**. Four test levels:

Test Level	Users	Duration	Goal
Smoke test	10	5 min	Basic sanity — nothing is broken
Load test	500 (2× peak)	30 min	P99 latency stays within target
Stress test	5,000 (10× peak)	15 min	Find the breaking point; verify HPA fires
Soak test	300	8 hours	Find memory leaks or connection pool exhaustion

What we check in each run:

- P99 < 1 second
- Error rate < 0.1%
- HPA scales pods from 2 → 20 as CPU rises
- No `OutOfMemoryError` after 8 hours
- No "too many connections" error from Aurora (proves RDS Proxy works)

中文解释：

压测4个阶段：

1. 冒烟（10用户/5分钟）：验证基本功能。
2. 负载（500用户/30分钟，2倍峰值）：验证P99达标。

3. 压力（5000用户，10倍峰值）：找到系统瓶颈，验证HPA自动扩容。
4. 稳定性（300用户/8小时）：排查内存泄漏和连接池耗尽。

重点指标：P99 < 1秒，错误率 < 0.1%，HPA正常触发，Aurora无连接超限报错。

Q37. How do you handle a sudden 10x traffic spike — like a big marketing campaign?

问题37：如果突然来了10倍流量（比如一个大型营销活动），你们怎么应对？

English Answer:

Two strategies: **reactive** (automatic) and **proactive** (planned ahead).

Reactive — Auto-scaling (always on):

- HPA detects CPU > 60%, adds pods in ~15 seconds (up to 20 pods per service)
- Karpenter detects "Pending" pods, adds new EC2 node in ~60 seconds
- For a 10x spike, HPA scales from 2 → 20 pods automatically

Proactive — Pre-warm before a known campaign:

1. Set `minReplicas: 10` in HPA 1 hour before the campaign starts
2. Add one Aurora read replica to handle read-heavy traffic
3. Increase Redis memory for higher cache throughput
4. CloudFront handles static assets at unlimited scale — no action needed

Safety net:

- WAF rate limiting: blocks any single IP sending > 1,000 requests in 5 minutes
- Circuit breakers prevent one slow service from cascading to others

中文解释：

被动（自动）：HPA 15秒内扩Pod（最多20个），Karpenter 60秒内加Node，10倍流量基本可以自动吸收。

主动（已知活动）：活动前提前把 `minReplicas` 调高，加 Aurora 读副本，扩 Redis 内存，避免冷启动延迟。

兜底：WAF 频率限制（1000次/5分钟/IP）+ 各服务间断路器防雪崩。

Q38. What is your database connection pool size? How did you decide on the number?

问题38：你们的数据库连接池大小是多少？怎么定的？

English Answer:

The math:

```
5 services × 20 pods (max) × 10 HikariCP connections = 1,000 connections to Aurora
Aurora db.r6g.large supports ~800–1,000 connections maximum
```

Without RDS Proxy, we would hit Aurora's connection limit during scale-out. That is why **RDS Proxy is mandatory**.

With RDS Proxy:

- All 1,000 pod connections go to RDS Proxy
- RDS Proxy maintains a fixed pool of 500 connections to Aurora
- Even if we scale to 50 pods, the Aurora connection count stays at 500

HikariCP settings per pod:

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10      # max 10 connections per pod
      minimum-idle: 2           # keep 2 warm connections always
      connection-timeout: 3000  # fail fast – 3 seconds, not hang forever
      idle-timeout: 300000      # close idle connections after 5 minutes
```

`connection-timeout: 3000` is important: if no connection is available, the request fails fast after 3 seconds instead of hanging forever and blocking a thread.

中文解释：

计算：5服务 × 20Pod × 10连接 = 1000，正好撞 Aurora 上限（约1000）。

RDS Proxy 的必要性：所有 Pod 连接指向 RDS Proxy，Proxy 维护到 Aurora 的固定连接池（500个），无论 Pod 怎么扩，Aurora 始终只看到500个连接，不会超限。

HikariCP 关键参数： `maximum-pool-size=10`（每Pod10个），`connection-timeout=3000`（3秒等不到连接直接抛异常，快速失败比无限等待强得多）。

Quick Reference — Key Technology Choices

快速参考：核心技术选型理由

Technology	Why We Chose It	Why Not the Alternative
EKS	Network Policy, PDB, full K8s ecosystem	ECS Fargate: less control, no Network Policy
Aurora PostgreSQL	30s failover, Global DB, auto storage	Regular RDS: slower failover, no Global DB
RDS Proxy	Connection pooling, transparent failover	Direct connection: connection limit, slow failover
IRSA	No long-lived credentials, per-service least privilege	Hard-coded AK/SK: security risk, key rotation pain
ArgoCD GitOps	Git as source of truth, drift detection, audit trail	kubectl apply: no audit, no drift detection
Secrets Store CSI	Secrets never stored in K8s Secret (base64 only)	K8s Secret: base64 is not encryption
CloudFront + WAF	Edge protection, DDoS absorption, geo-restriction	Direct ALB exposure: no edge protection
Canary Deployment	Max 10% users affected if a bug is found	Big-bang: 100% users affected

面试小贴士：

- 每个技术选型都要说"为什么选它，为什么不选另一个"（trade-off）。
- 两岸三中心优先讲 RTO/RPO 数字，面试官印象深刻。
- 容量题不要背数字，要说推导过程：用户数 → DAU → 并发 → QPS。
- 遇到不会的，说"In our current design we did X, but if I had more time I would also consider Y"。

Part 9 — Java+React Full Stack Developer (JD 定向)

第九部分：Java+React 全栈开发（针对新 JD）

以下题目直接对应 Java+React 全栈开发岗 JD 要求。React 相关只涉及 React.js，不涉及 Vue 或 Angular。

Q39. What is the difference between Spring Boot and Spring Cloud? When do you need Spring Cloud?

问题39：Spring Boot 和 Spring Cloud 的区别是什么？什么时候需要 Spring Cloud？

English Answer:

Spring Boot is a framework to build a single application quickly. It gives you auto-configuration, embedded servers, and production-ready features (actuator, health checks) out of the box.

Spring Cloud is a set of tools to help many microservices work together in a distributed system. You need Spring Cloud when you have many services that must call each other.

Problem	Spring Cloud Component	What It Does
Service A calls Service B	OpenFeign	Simple REST client — like writing a local interface
Service B is down	Resilience4j CircuitBreaker	Stop failures from spreading to other services
Too many calls to Service B	@Retryable	Automatically retry failed calls
Each service has a different config	Spring Cloud Config	Centralized config server — all configs in one place
Finding other services	Spring Cloud Netflix Eureka	Service registry — like a phone book for services
API Gateway	Spring Cloud Gateway	Single entry point, routing, rate limiting

When you need it: When you have more than 3-5 microservices that call each other. For a simple 2-service app, Spring Boot alone is enough.

中文解释：

- **Spring Boot**：快速构建单个微服务，内嵌服务器，开箱即用的健康检查。
- **Spring Cloud**：协调多个微服务的工具集，用于解决分布式系统特有的问题。

典型场景：服务间调用（OpenFeign）、防雪崩（Resilience4j）、配置中心（Config Server）、服务注册发现（Eureka）、API网关（Gateway）。服务数量超过3-5个时，才需要引入Spring Cloud。

Q40. What is Spring Batch? When would you use it instead of a normal Spring Boot service?

问题40：什么是 Spring Batch？什么时候用它而不是普通的 Spring Boot 服务？

English Answer:

Spring Batch is a framework for processing **large volumes of data** in a reliable way. It is designed for batch jobs — tasks that run on a schedule, process millions of records, and don't

need human interaction.

Normal Spring Boot service = handle one request at a time (online/OLTP). **Spring Batch job** = process millions of records overnight (offline/OLAP).

Example in our system: Every night at 2am, Spring Batch:

1. Reads 500,000 user portfolio snapshots from Aurora
2. Calculates daily P&L (profit and loss) for each user
3. Writes results back to the database
4. Sends email summaries to users

Key Spring Batch concepts:

Concept	Meaning
Job	One batch task (e.g., "calculate daily P&L")
Step	One phase of a job (read → process → write)
ItemReader	Reads data from a source (database, file, API)
ItemProcessor	Transforms each record
ItemWriter	Writes the processed record to destination
Chunk	Process N records in one transaction (e.g., chunk=100)

Why not just use a scheduled Spring Boot `@Scheduled` method?

For 100 records, `@Scheduled` is fine. For 5 million records, you need:

- Restartable jobs (if the server crashes at record 3 million, resume from there)
- Chunk-based processing (commit every 100 records, not all 5 million)
- Skip policy (skip bad records, don't fail the whole job)
- Monitoring and restart UI

中文解释：

Spring Batch 用于处理**大批量离线数据**，典型场景：每日凌晨跑批计算用户收益、同步大批量数据、定时报表生成。

核心概念：**Job**（一个批处理任务）→ **Step**（读→处→写三阶段）→ **Chunk**（每N条提交一次事务）。重启可恢复（服务器崩了从3百万条Resume）、跳过异常行（脏数据不导致整批失败），这些是普通 `@Scheduled` 方法做不到的。

Q41. Explain Hibernate caching — first-level vs second-level cache. How do you avoid stale data?

问题41：解释 Hibernate 的一级缓存和二级缓存。如何避免脏数据？

English Answer:

Hibernate sits between your Java code and the database. It uses caching to reduce database round-trips.

First-Level Cache (L1 Cache):

- Attached to the **current Hibernate Session** (one database transaction).
- Automatically enabled. Cannot be turned off.
- When you `session.find(User.class, 1)` twice in the same transaction, the second call hits L1 cache — zero database queries.
- Once the session closes, L1 cache is gone.

Second-Level Cache (L2 Cache):

- Attached to the **SessionFactory** — shared across all sessions (entire application).
- Must be explicitly enabled manually. Uses EhCache, Redis, or Infinispan.
- Caches **entities** (entire rows). When any session loads User #1, it's cached for all future sessions.
- Has a TTL (time-to-live) and a max size.

```
// Enable L2 cache
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=ehcache

// Mark an entity as cacheable
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Fund { ... }
```

The old data problem:

If User A changes their email, and User B reads the same User entity from L2 cache 5 seconds later, User B still sees the old email — until the cache expires.

Solutions:

1. **Short TTL** — set cache expiry to 30-60 seconds (acceptable for most use cases)
2. **Remove from cache when updated** — when entity is updated, delete it from cache immediately
3. **READ_WRITE strategy** — Hibernate writes to cache on update, so next read gets fresh data
4. **For financial data: never cache writes** — disable L2 cache for Order/Portfolio entities

中文解释：

- **一级缓存**：绑定到Session，事务内有效，自动开启，不可关闭，同一事务内重复查询不走数据库。
- **二级缓存**：绑定到SessionFactory，全应用共享，需手动开启，缓存实体（整行数据），有TTL过期策略。

脏数据问题（User A改了数据，User B从缓存读到旧数据）：设置短TTL（30-60秒）、写操作时主动失效缓存（`session.evict()`）、金融关键数据（订单/持仓）禁用L2缓存。

Q42. What is the difference between `@Transactional` and `@Transactional(readOnly = true)` ? When does each matter?

问题42：`@Transactional` 和 `@Transactional(readOnly = true)` 的区别是什么？各在什么场景下使用？

English Answer:

Both wrap a method in a database transaction, but with different database access hints.

`@Transactional` (default, `readOnly`):

- The database connection is opened with **write intent** (意图) .
- Hibernate session dirty check is active — any changed entity is automatically flushed to the database at commit.
- Uses a standard connection from the pool.
- **Use when:** You are inserting, updating, or deleting data.

`@Transactional(readOnly = true)` :

- The database connection is opened with **read intent**.
- Hibernate **skips dirty checking** — it does not track entity changes (saves CPU).

- Some databases (PostgreSQL) use a **read replica** if one is available — reduces load on the primary.
- The database may optimize its query plan for read-only workloads.
- **Use when:** You are only reading data (select queries, reports, data export).

```
// Read-only query – faster, no dirty checking
@Transactional(readOnly = true)
public List<Fund> getAllFunds() {
    return fundRepository.findAll();
}

// Write operation – needs dirty checking
@Transactional
public Order placeOrder(OrderRequest request) {
    Order order = new Order();
    order.setStatus(OrderStatus.PENDING);
    return orderRepository.save(order); // must be written
}
```

Performance difference: For a read-only query on 10,000 rows, `readOnly=true` can be 10-20% faster because Hibernate skips the dirty-check step.

中文解释：

- `@Transactional`：默认读写事务，Hibernate开启脏数据检查（扫描所有托管实体是否有变更），适用于增删改操作。
- `@Transactional(readOnly = true)`：只读事务，跳过脏数据检查（节省CPU），部分数据库会路由到读副本（减轻主库压力），适合查询报表、列表页等只读场景。

性能收益：查询万级数据时，`readOnly=true` 可提升10-20%性能。

Q43. What is a design pattern you have used to make code more reusable? Give a concrete example.

问题43： 你用过哪种设计模式来提高代码复用率？请举例说明。

English Answer:

I will give two examples from real work.

Example 1 — Template Method Pattern for REST API controllers:

Every controller in our system does the same 4 steps: validate input → call service → handle exception → return response. Instead of repeating this in every controller, we use a base class:

```
public abstract class BaseApiController<T, R> {
    protected abstract T serviceCall(R request);    // subclasses implement
    this

    public ApiResponse<T> handle(R request) {
        validate(request);        // same for all
        T result = serviceCall(request); // different per controller
        return ApiResponse.ok(result); // same for all
    }
}

@RestController
@RequestMapping("/api/v1/orders")
public class OrderController extends BaseApiController<OrderDto, OrderRequest>
{
    @Override
    protected OrderDto serviceCall(OrderRequest request) {
        return orderService.createOrder(request);
    }
}
```

Now if we need to add request logging or audit, we only change one place.

Example 2 — Strategy Pattern for fee calculation:

Different fund products charge fees differently. Instead of if-else:

```

public interface FeeCalculationStrategy {
    Money calculate(Order order, Fund fund);
}

@Service
public class PercentageFeeStrategy implements FeeCalculationStrategy {
    public Money calculate(Order order, Fund fund) {
        return order.getAmount().multiply(fund.getFeeRate());
    }
}

@Service
public class FlatFeeStrategy implements FeeCalculationStrategy {
    public Money calculate(Order order, Fund fund) {
        return fund.getFlatFee();
    }
}

// Inject the right strategy
@Autowired
private Map<String, FeeCalculationStrategy> strategies; // Spring auto-injects all

public Money computeFee(Order order, Fund fund) {
    FeeCalculationStrategy strategy = strategies.get(fund.getFeeType());
    return strategy.calculate(order, fund);
}

```

Now adding a new fee type only requires adding a new class, not modifying existing code.

中文解释：

模板方法模式：所有Controller都执行"校验→调用Service→处理异常→返回"，抽象到BaseController，新Controller只需实现具体Service调用逻辑，公共逻辑一处修改全局生效。

策略模式：不同基金产品费率计算方式不同（按比例/固定额），定义FeeCalculationStrategy接口，每种策略一个实现类，Spring自动注入所有策略到Map，按fund类型动态选取。添加新品种只需新增策略类，不动现有代码（开闭原则）。

Q44. How do you handle database transactions that span multiple services (distributed transactions)?

问题44：如何处理跨多个服务的数据库事务（分布式事务）？

English Answer:

In a micro services system, one business operation often touches multiple databases. A "transfer" operation needs to debit Account A and credit Account B — but Account A is in `user-service` and Account B is in `order-service`. You cannot use a single ACID transaction.

Two main approaches:

Approach 1 — Saga Pattern (preferred for most cases):

The operation is broken into a sequence of local transactions. Each step publishes an event; the next step listens and proceeds. If any step fails, **undo steps** reverse the previous steps.

Happy path:

Step 1: Reserve funds (local DB commit)	→ publishes ReserveCompleted
Step 2: Create order (local DB commit)	→ publishes OrderCreated
Step 3: Confirm transfer (local DB commit)	→ publishes TransferConfirmed

Failure at Step 2:

Compensation: Undo Step 1 (refund the reservation)

In our system, we use **event-based Saga** with Kafka events:

- Each service owns its local transaction
- Each service publishes events that trigger the next service
- If a service fails, it publishes a compensating event

Approach 2 — Two-Phase Commit (2PC) (rarely used):

A coordinator asks all services to "prepare." If all say yes, the coordinator tells them all to "commit." If any says no, all roll back.

Problem: If the coordinator crashes after "prepare" but before "commit," services are stuck waiting. Also, all databases must support XA (not all do). We avoid 2PC in our architecture.

中文解释:

跨服务事务不能用本地ACID事务，需要分布式事务方案：

Saga模式（我们采用）：将操作拆成一系列本地事务，每步成功则发布事件触发下一步，失败则执行补偿事务（撤销前几步）。用Kafka事件实现 choreography-based Saga，详见上文示例。

两阶段提交（2PC）：协调者先问所有服务"准备好了吗"，全部确认后才"提交"。缺点：协调者崩溃时服务卡死；需要数据库支持XA协议。实际生产中极少使用。

Q45. How do you connect a React frontend to a Spring Boot backend? Walk me through the flow.

问题45：React 前端如何连接 Spring Boot 后端？请描述整个调用链路。

English Answer:

Here is the full flow from React button click to Spring Boot response:

Step 1 — React component calls the API:

```
// React: call the backend API
const [funds, setFunds] = useState([]);

useEffect(() => {
  fetch('http://localhost:8080/api/v1/funds')
    .then(res => res.json())
    .then(data => setFunds(data));
}, []);

return funds.map(fund => <FundCard key={fund.code} fund={fund} />);
```

Step 2 — Spring Boot Controller receives the request:

```
@RestController
@RequestMapping("/api/v1/funds")
@RequiredArgsConstructor
public class FundController {
    private final FundService fundService;

    @GetMapping
    public ResponseEntity<List<FundDto>> getAllFunds() {
        List<FundDto> funds = fundService.getAllFunds();
        return ResponseEntity.ok(funds); // returns JSON
    }
}
```

Step 3 — CORS configuration (Spring Boot must allow React):


```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000") // React dev server
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*");
    }
}
```

Step 4 — React sends the request with JWT token:

```
const response = await fetch('http://localhost:8080/api/v1/orders', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ' + jwtToken // JWT from login
  },
  body: JSON.stringify(orderRequest)
});
```

Step 5 — Spring Security validates the JWT and extracts user info:

```
SecurityFilterChain {
    http.addFilterBefore(jwtAuthFilter,
        UsernamePasswordAuthenticationFilter.class)
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/funds/**").permitAll()
            .requestMatchers("/api/v1/orders/**").authenticated()
        );
}
```

Step 6 — Response returns JSON to React:

```
HTTP 200 OK
Content-Type: application/json

[
  { "code": "FUND_A", "name": "Global Tech Fund", "price": 12.50 },
  { "code": "FUND_B", "name": "Bond Saver", "price": 9.80 }
]
```

中文解释：

React调用Spring Boot后端的完整链路：

1. **React:** `fetch('/api/v1/funds')` 发起HTTP请求
 2. **Spring Boot:** `@RestController` 接收请求，调用Service层，返回JSON
 3. **CORS配置:** 后端必须允许前端域名（开发环境 `localhost:3000`）
 4. **JWT认证:** 敏感API需要在请求头带 `Authorization: Bearer <token>`
 5. **Spring Security:** 过滤器链验证JWT token，确认用户身份
 6. 返回: 后端返回JSON，前端用 `res.json()` 解析并更新UI状态
-

Q46. What is the difference between `useState` and `useEffect` in React? Give an example.

问题46：React 中 `useState` 和 `useEffect` 的区别是什么？请举例。

English Answer:

`useState` = stores data that changes over time (like a variable, but changing it causes the component to re-render).

`useEffect` = runs extra actions after the component renders (like fetching data, setting up timers, subscribing to events).

```
function FundList() {
  // useState: stores the list of funds
  const [funds, setFunds] = useState([]); // funds = [], setFunds is the
  updater

  // useEffect: runs AFTER the first render (and whenever deps change)
  useEffect(() => {
    // Fetch data from backend when component mounts
    fetch('http://localhost:8080/api/v1/funds')
      .then(res => res.json())
      .then(data => setFunds(data));

    // Cleanup when component is removed
    return () => {
      console.log('Component will unmount - clean up');
    };
  }, []); // [] = empty watch list = run only once after first render

  return (
    <div>
      {funds.map(fund => (
        <FundCard key={fund.code} name={fund.name} price={fund.price}
      ))}
    </div>
  );
}
```

Three ways to write `useEffect` with watch list:

Syntax	When it runs
<code>useEffect(fn, [])</code>	Only once after first render (good for <code>fetch</code>)
<code>useEffect(fn, [x])</code>	After first render AND every time <code>x</code> changes
<code>useEffect(fn)</code>	After every render (rarely what you want — avoid)

Common mistake: Putting an object in the watch list can cause an infinite loop because a new object is created on every render. Use primitive values or use `useMemo` .

中文解释:

- `useState` : 声明组件内部状态, 状态变化触发组件重新渲染 (类似Vue的 `data()`) 。
- `useEffect` : 副作用钩子, 在组件渲染后执行, 常用于数据获取、订阅、定时器。

常见用法：

- `useEffect(fn, [])`：只在首次渲染后执行（适合 `fetch` 初始化数据）
- `useEffect(fn, [count])`：首次渲染 + `count` 变化时执行
- `useEffect(fn)`：每次渲染都执行（慎用，可能导致无限循环）

常见错误：把对象放入依赖数组（每次渲染创建新对象），会导致无限循环。依赖数组应放原始类型值。

Q47. What is React.memo and when would you use it?

问题47：React.memo 是什么？什么时候应该使用它？

English Answer:

React.memo is a performance optimization. It caches a component's output. If the component's **props don't change**, React reuses the last rendered result instead of re-rendering the component.

Without React.memo:

```
function FundRow({ fund }) {  
  // This re-renders every time ANY parent component re-renders  
  return <tr><td>{fund.name}</td><td>{fund.price}</td></tr>;  
}
```

With React.memo:

```
const FundRow = React.memo(function FundRow({ fund }) {  
  // Only re-renders if `fund` prop actually changes  
  return <tr><td>{fund.name}</td><td>{fund.price}</td></tr>;  
});
```

When to use it:

- The component renders **frequently** (e.g., a table with 100 rows, each row re-renders on parent state change)
- The component does **heavy computation** (chart rendering, data processing)
- The component receives **the same props repeatedly** (list items in a stable list)

When NOT to use it:

- The component is **lightweight** (memoization overhead > rendering cost)
- Props **always change** (cached version is never reused, just extra work with no benefit)
- Don't use it as a default — measure first with React DevTools Profiler

中文解释:

React.memo 是一个性能优化工具，可以缓存组件渲染结果。如果组件的props没有变化，React会跳过渲染，直接复用上次的结果。

适用场景：表格行组件（父组件状态变化时避免所有行都重渲染）、做复杂计算的子组件（图表渲染）、列表项（props稳定不变时效果最好）。

禁用场景：组件本身很轻量（memo的开销反而更大）、props每次都变（缓存命中率为0）。

Q48. How do you manage state in React when you have many components that need to share data?

问题48：React 中多个组件需要共享状态时，你们怎么管理？

English Answer:

There are three main approaches, depending on how widely the data needs to be shared:

1. Props drilling (simple, for small apps):

```
// Pass data from parent to deeply nested child via props
<App>
  <Header user={user} />           // passes user down
  <Nav user={user} />              // passes user down again
  <Profile user={user} />          // finally uses user
```

Problem: If `Profile` is 5 levels deep and `App` is the only place with `user`, you pass `user` through 4 intermediate components that don't need it.

2. Context API (medium complexity, for shared data):

```
// Create a context
const UserContext = createContext(null);

// Provide it at the top level
function App() {
  const [user, setUser] = useState(currentUser);
  return (
    <UserContext.Provider value={{ user, setUser }}>
      <Dashboard />
    </UserContext.Provider>
  );
}

// Use it in any nested component – no props needed
function Profile() {
  const { user, setUser } = useContext(UserContext);
  return <div>Hello {user.name}</div>;
}
```

Best for: logged-in user data, theme, language preference — data that many unrelated components need.

3. State management library like Zustand (complex apps):

For large apps with many entities and complex interactions:

```
// Store definition
const useOrderStore = create((set) => ({
  orders: [],
  addOrder: (order) => set(state => ({ orders: [...state.orders, order] })),
}));

// In any component
function OrderList() {
  const { orders, addOrder } = useOrderStore();
  return orders.map(o => <OrderRow key={o.id} order={o} />);
}
```

中文解释：

React状态共享三板斧：

1. **Props逐层传递**：简单，但多层嵌套时中间层被迫接收不需要的props（prop drilling），维护噩梦。

2. **Context API**: 跨组件共享全局数据（用户登录态、主题、语言），在顶级Provider注入，子组件随时 `useContext` 取用，适合中等复杂度应用。
3. **Zustand等状态管理库**: 复杂应用（多实体、频繁交互），集中管理所有状态，组件直接订阅，状态变更自动触发相关组件重渲染。

金融系统典型用法：用户登录态用Context，订单列表/持仓数据用Zustand。

Q49. How do you handle form validation in React and Spring Boot at the same time?

问题49：React 和 Spring Boot 的表单校验如何同时处理？

English Answer:

We validate on **both sides** — client-side for UX (instant feedback) and server-side for security (never trust the client).

React — Client-side validation:

```

function OrderForm() {
  const [amount, setAmount] = useState('');
  const [error, setError] = useState('');

  const validate = (value) => {
    const num = parseFloat(value);
    if (!value) return 'Amount is required';
    if (isNaN(num)) return 'Must be a number';
    if (num < 100) return 'Minimum amount is 100';
    if (num > 1000000) return 'Maximum amount is 1,000,000';
    return '';
  };

  return (
    <div>
      <input
        value={amount}
        onChange={e => {
          setAmount(e.target.value);
          setError(validate(e.target.value)); // validate on change
        }}
      />
      {error && <span style={{color:'red'}}>{error}</span>}
    </div>
  );
}

```

Spring Boot — Server-side validation (the real check):

```

@PostMapping("/orders")
public ResponseEntity<OrderResponse> placeOrder(
    @Valid @RequestBody @NotNull OrderRequest request,
    BindingResult result) {

  if (result.hasErrors()) {
    return ResponseEntity.badRequest()
      .body(OrderResponse.error(result.getFieldErrors()));
  }
  return ResponseEntity.ok(orderService.createOrder(request));
}

```



```
// Validation annotations on the DTO
public class OrderRequest {
    @NotNull(message = "Amount is required")
    @DecimalMin(value = "100.00", message = "Minimum amount is 100")
    @DecimalMax(value = "1000000.00", message = "Maximum is 1,000,000")
    private BigDecimal amount;

    @NotBlank(message = "Fund code is required")
    private String fundCode;
}
```

Why both sides?

- Client-side: better UX, instant feedback, no network round-trip
- Server-side: **security** — a bad user can bypass React DevTools and send any payload directly to the API. Server-side validation is the real wall.

中文解释：

两端校验，缺一不可：

- **前端校验**：改善用户体验，输入时即时反馈，不需要等网络往返。
- **后端校验**：唯一可信数据源，恶意用户可以直接用curl/Postman绕过前端发送任意数据，后端校验才是真正的安全防线。

Spring Boot用 @Valid 注解+Bean Validation注解（@NotNull、@DecimalMin）自动校验请求体，校验失败返回400和详细错误信息。

Q50. How do you Dockerize a Spring Boot application? Walk me through the Dockerfile.

问题50：如何将 Spring Boot 应用 Docker 化？请讲解 Dockerfile 的写法。

English Answer:

Here is a standard multi-stage Dockerfile for a Spring Boot application:

```

# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app
# Copy pom.xml first (layer caching – Maven deps are downloaded only when
pom.xml changes)
COPY pom.xml .
RUN mvn dependency:go-offline # download all dependencies
COPY src ./src
RUN mvn package -DskipTests # build JAR, skip tests in build stage

# Stage 2: Runtime – small image, only the JAR
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
# Copy only the JAR from the build stage
COPY --from=builder /app/target/myapp.jar app.jar
# Copy config (can be overridden at runtime with volume mount)
COPY src/main/resources/application-prod.yml config/
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar", "--spring.config.location=config/"]

```

Key decisions explained:

Line	Why
maven:3.9-eclipse-temurin-21	Full Maven for building (JDK 21 matches production)
Multi-stage build	Build stage is heavy; runtime stage uses slim JRE image — final image is ~200MB
Layer caching	pom.xml copied first, then source — Maven deps cached, fast rebuilds
eclipse-temurin:21-jre-alpine	Alpine Linux JRE — tiny image (~80MB), smaller attack surface
--from=builder	Only the JAR is copied to runtime stage; build tools are excluded

Build and run commands:

```
# Build
docker build -t smart-invest/user-service:1.0.0 .

# Run locally (simulating production config)
docker run -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e DATABASE_URL=jdbc:postgresql://host.docker.internal:5432/smartinvest \
  smart-invest/user-service:1.0.0
```

中文解释：

Dockerfile核心要点：

- **多阶段构建**：Build阶段用Maven+JDK 21打包，Runtime阶段用精简JRE镜像（Alpine Linux，约80MB），最终镜像比Build阶段小得多。
- **层缓存优化**：`pom.xml` 先复制、先下载依赖，再复制源码——pom不变时构建命中缓存，速度极快。
- **最小权限镜像**：Alpine Linux + JRE（非JDK），攻击面最小化。
- **`--from=builder`**：只把JAR文件复制到运行镜像，Build工具不进入最终镜像。

Q51. What is the difference between Docker `COPY` and `ADD` ? When would you use each?

问题51：Dockerfile 中 `COPY` 和 `ADD` 的区别是什么？各在什么场景使用？

English Answer:

Both copy files from the host into the Docker image, but with important differences:

Instruction	What it does	When to use
<code>COPY</code>	Copies files/directories from the build context	Preferred for most cases
<code>ADD</code>	Copies files AND can extract tar files AND fetch URLs	Only for .tar extraction or remote URLs

```
# Preferred: simple file copy
COPY target/myapp.jar /app/app.jar
COPY config/ /app/config/

# Only use ADD for these two special cases:
ADD app.tar.gz /app/          # auto-extracts tar.gz into /app/
ADD https://example.com/file.zip /app/ # downloads and copies
```

Why prefer COPY?

- **COPY** is **explicit** — you always know exactly what you're copying.
- **ADD** has hidden behavior (auto-extraction) that can cause confusion.
- **ADD** can fetch from URLs, which makes the build context less predictable.
- Security scanners work better with **COPY** because the source is always local.

Best practice: Use **COPY** 99% of the time. Use **ADD** only when you genuinely need tar extraction.

中文解释:

- **COPY** : 简单复制, 推荐使用, 行为可预测。
- **ADD** : 除了复制, 还能自动解压tar.gz文件, 或从远程URL下载文件。

大多数场景用 **COPY** 即可。只有需要解压tar包时才用 **ADD** (**ADD app.tar.gz /app/**)。 **ADD** 从URL下载文件容易引入不确定性和安全风险, 慎用。

Q52. You have a memory leak in a Java application running in Docker. How do you diagnose it?

问题52: Java 应用在 Docker 中出现了内存泄漏, 你如何排查?

English Answer:

Here is my step-by-step diagnosis process:

Step 1 — Check if it's really a memory leak (not just normal high usage):

```
# Check container memory usage – is it growing continuously?
docker stats --no-stream smart-invest-user-service

# Check container memory limit
docker inspect smart-invest-user-service | grep Memory
# If memory usage hits the limit → OOMKilled (exit code 137)
```

Step 2 — Enable JVM heap dumps in Docker:

```
# docker-compose or K8s pod spec
env:
  - name: JAVA_OPTS
    value: "-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps/ -XX:+UseG1GC"
```

Step 3 — Get a heap dump from inside the container:

```
# Find the JVM PID inside the container
docker exec smart-invest-user-service jcmd | grep java

# Trigger a heap dump (if OOM already happened, it's already in /dumps/)
docker exec smart-invest-user-service jcmd <pid> GC.heap_dump /dumps/heap.hprof

# Copy it out of the container
docker cp smart-invest-user-service:/dumps/heap.hprof ./heap.hprof
```

Step 4 — Analyze the heap dump:

```
# Use Eclipse Memory Analyzer (MAT) – free tool
# Open heap.hprof in MAT and look for:
# 1. "Leak Suspects" – objects that are suspiciously large
# 2. "Top Consumers" – biggest objects by shallow size
# 3. "Dominator Tree" – who holds the biggest object graph
```

Step 5 — Common Java memory leak causes in Spring Boot:

Cause	Symptom	Fix
Unclosed EntityManager / Session	Hibernate session never closed	Use <code>@Transactional</code> , always close in <code>finally</code>
Static <code>Map</code> accumulating data	Cache grows forever without eviction	Use <code>Cache eviction</code> or <code>TTL</code>
<code>ThreadLocal</code> not cleaned up	Thread pool reused, but <code>ThreadLocal</code> holds references	Always call <code>ThreadLocal.remove()</code> in filter
Large list without pagination	Query returns 10M rows, OOM	Add <code>Pageable</code> to repository methods

中文解释：

Java内存泄漏排查步骤：

1. 确认是泄漏： `docker stats` 观察内存是否持续增长到上限
2. 开启堆转储： JVM参数 `-XX:+HeapDumpOnOutOfMemoryError` 让OOM时自动生成 `.hprof` 文件
3. 导出分析： 用 `jcmd <pid> GC.heap_dump` 手动触发，用Eclipse MAT工具打开分析"泄漏嫌疑人"和"最大消耗对象"
4. 常见原因： EntityManager未关闭、静态Map无限累积、ThreadLocal未清理、大查询无分页

Q53. How do you write a JUnit 5 test that verifies a method throws an exception correctly?

问题53： 如何用 JUnit 5 写一个测试，验证方法正确抛出了异常？

English Answer:

In JUnit 5, you use `assertThrows` from `org.junit.jupiter.api.Assertions` :

```

import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;

    @InjectMocks
    private OrderService orderService;

    @Test
    void shouldThrowExceptionWhenAmountIsBelowMinimum() {
        // Setup
        OrderRequest request = new OrderRequest();
        request.setAmount(new BigDecimal("50")); // minimum is 100

        // Call and check – assertThrows catches the exception
        OrderValidationException ex = assertThrows(
            OrderValidationException.class, // expected exception type
            () -> orderService.placeOrder(request) // call the method
        );

        // Also verify the exception message
        assertEquals("Amount must be at least 100", ex.getMessage());
    }

    @Test
    void shouldThrowExceptionWhenFundCodeIsBlank() {
        OrderRequest request = new OrderRequest();
        request.setAmount(new BigDecimal("500"));
        request.setFundCode(null);

        OrderValidationException ex = assertThrows(
            OrderValidationException.class,
            () -> orderService.placeOrder(request)
        );

        assertTrue(ex.getMessage().contains("fund code"));
    }
}

```

Other useful JUnit 5 assertions:

```
// Test that an exception IS NOT thrown (happy path verification)
assertDoesNotThrow(() -> orderService.placeOrder(validRequest));

// Test multiple exceptions in the same test
assertAll(
    () -> assertThrows(ValidationException.class, () -> method(null)),
    () -> assertThrows(ValidationException.class, () -> method(""))
);

// Test that code takes too long (performance test)
assertTimeout(Duration.ofMillis(100), () -> service.fastOperation());
```

中文解释：

JUnit 5 用 `assertThrows` 捕获并验证异常：

- **第一个参数**：期望的异常类型
- **第二个参数**：lambda表达式调用被测方法
- **assertEquals**：进一步验证异常消息内容

常用变体：`assertDoesNotThrow`（验证正常路径不抛异常）、`assertAll`（一个测试验证多个断言）、`assertTimeout`（验证执行时间不超时）。

Q54. What is the difference between mocking with `@Mock` and `@MockBean` in Spring Boot testing?

问题54：Spring Boot 测试中 `@Mock` 和 `@MockBean` 的区别是什么？

English Answer:

Both create fake (mock) objects, but they serve different purposes.

	@Mock (Mockito)	@MockBean (Spring Boot)
Framework	Mockito	Spring Boot Test
Scope	Unit test — only the test class	Integration test — Spring context
Use when	Testing a single service in isolation	Replacing a real Spring bean in the application context
Spring context	Does NOT start Spring context	Starts full Spring context with mocked bean

@Mock — Unit test (fast, no Spring):

```

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock
    private FundClient fundClient;      // fake – no real HTTP call

    @Mock
    private OrderRepository orderRepository; // fake – no real DB

    @InjectMocks
    private OrderService orderService;    // real service with mocks injected

    @Test
    void shouldPlaceOrder() {
        when(fundClient.getPrice("FUND_A")).thenReturn(new
        BigDecimal("12.50"));
        // ... test real business logic, no Spring needed
    }
}

```

@MockBean — Integration test (slower, but full Spring context):

```

@SpringBootTest
@AutoConfigureMockMvc
class OrderControllerIntegrationTest {
    @MockBean
    private FundClient fundClient; // replaces the real FundClient in Spring
    context

    @Autowired
    private MockMvc mockMvc; // test HTTP layer

    @Test
    void shouldReturn400WhenValidationFails() throws Exception {
        when(fundClient.getPrice(any())).thenReturn(new BigDecimal("12.50"));
        mockMvc.perform(post("/api/v1/orders")
            .contentType("application/json")
            .content("{\"amount\": 50}"))
            .andExpect(status().isBadRequest()); // 400
    }
}

```

Rule of thumb: Start with `@Mock` (unit tests) for business logic. Use `@MockBean` when you need to test how the HTTP layer interacts with your service.

中文解释:

- `@Mock` : Mockito提供, 只在当前测试类生效, 不启动Spring容器, 速度快, 用于纯业务逻辑单元测试。
- `@MockBean` : Spring Boot Test提供, 替换Spring容器中的真实Bean, 启动完整Spring上下文, 速度较慢, 用于测试HTTP层和Spring Bean的集成。

实际选择: 业务逻辑用 `@Mock` (快, 隔离), Controller/Service集成测试用 `@MockBean` (需要真实Spring上下文)。

Q55. What is the difference between Scrum and Kanban? Which do you prefer and why?

问题55: Scrum 和 Kanban 的区别是什么? 你更倾向于哪个?

English Answer:

Both are Agile frameworks, but they work very differently.

Aspect	Scrum	Kanban
Cadence / Rhythm	Fixed sprints (2-3 weeks)	Continuous flow, no fixed sprints
Roles	Scrum Master, Product Owner, Dev Team	No required roles — just WIP limits
Planning	Sprint planning at start of sprint	Prioritize continuously
Change during sprint	Avoid — sprint is committed	Can change anytime — WIP limits protect flow
Best for	Product development with predictable releases	Operations/maintenance teams, frequent interruptions
Board	3 columns: To Do / In Progress / Done	Custom columns + explicit WIP limits per column

In our team:

We use **Scrum** for new feature development:

- 2-week sprints
- Sprint planning: "What can we commit to finishing this sprint?"
- Daily standup: "What did I do? What will I do? Any blockers?"
- Sprint review: demo to product owner
- Retrospective: "How can we improve?"

We use **Kanban** for production support (bug fixes, hotfixes):

- Bugs go directly into the "In Progress" column
- WIP limit of 3 bugs per developer
- When a developer finishes a feature, they pick up the next bug

中文解释:

维度	Scrum（迭代式）	Kanban（流动式）
周期	固定冲刺（2-3周）	持续流动，无固定冲刺
角色	Scrum Master、PO、开发团队	无强制角色，靠WIP限制管理
变更处理	冲刺内尽量不变更	随时可以插入新任务
适用场景	新功能开发，需要可预测的发布节奏	运维/支持团队，频繁被打断的场景

我们团队：新产品开发用Scrum（有承诺有节奏），生产支持/紧急修复用Kanban（灵活响应）。

Q56. What does a typical Code Review look like in your team? What do you look for as a reviewer?

问题56：你们团队的 **Code Review** 是怎么做的？作为 **reviewer** 你重点看什么？

English Answer:

In our team, every change requires at least **one approval** before merging to `main`. Here's our process:

As a PR author, I make sure to:

1. Keep the PR small — one feature or one bug fix, max ~400 lines changed
2. Write a clear description: "What changed, why, and how to test it"
3. Attach screenshots if it changes UI
4. Self-review the diff before asking others — read it as if you're the reviewer

As a reviewer, I check five things:

1. Correctness — does it do what it says?

```
// Red flag: method name says "get" but it modifies state
public User getUser(String id) {
    this.lastAccessTime = now(); // ❌ Changes state inside a "get" method
    return userRepository.findById(id);
}
```

2. Security — is there a vulnerability?

```
// Red flag: SQL injection possibility
@Query("SELECT u FROM User u WHERE u.name = '" + input + "'") // ❌ String
concat
// Fix: use parameter binding
@Query("SELECT u FROM User u WHERE u.name = :name")
```

3. Test coverage — is it tested?

- Every new method needs at least one unit test
- Happy path AND at least one error path

4. Design — is the code in the right place?

- Is the logic in the right service/class?
- Is the database operation in the repository layer, not the controller?

5. Performance — will it cause problems at scale?

- N+1 query (looping through a list and calling `findById` inside the loop)
- Missing database index for a query on a large table

中文解释：

PR作者：PR要小（一个功能或一个Bug，最多~400行）、描述清晰（改了什么、为什么改、怎么测试）、附截图、自审一遍。

Reviewer 五维检查：正确性（方法名和实现是否一致）、安全性（SQL注入、敏感数据日志）、测试覆盖（新增逻辑必须有测试）、设计（逻辑是否放在正确的层次）、性能（N+1查询、缺失索引）。

一条重要的reviewer原则：如果review意见超过10条，说明PR太大，应该拆分成更小的PR。

Part 10 — Message Queue Deep Dive（MQ 消息队列深度题）

第十部分：消息队列核心问题

这五道题是 MQ 生产实践中必须回答的问题，也是面试高频题。每道题都配英文和中文答案。

Q10-1. How do you guarantee that a message is reliably delivered? What if the broker crashes?

问题10-1：如何保证消息可靠投递？Broker 崩溃怎么办？

English Answer:

Reliable delivery has three parts — the producer side, the broker side, and the consumer side.

Producer side — don't lose the message before it reaches the broker:

1. **Use synchronous send with retry.** After calling `send()`, wait for an `ACK` from the broker. If you get a timeout or error, retry with exponential backoff.
2. **Use a local outbox table (Transactional Outbox Pattern).** Write the message to a database table in the same transaction as your business data. A separate thread (or CDC tool like Debezium) reads the outbox table and sends to the broker. This removes the "dual write" problem — you never have a case where the DB commit succeeded but the message was lost.

```
// Transactional Outbox - write message and business data in one DB transaction
@Transactional
public void placeOrder(Order order) {
    orderRepository.save(order);
    outboxRepository.save(new OutboxEvent("OrderPlaced", order.getId(),
    toJson(order)));
    // Outbox relay will pick this up and send to Kafka/MQ
}
```

Broker side — don't lose the message inside the broker:

- **Kafka:** Set `acks=all` (all in-sync replicas must confirm). Set `replication.factor=3`, `min.insync.replicas=2`. Data is written to disk before ACK.
- **IBM-MQ:** Use **persistent messages** (`MQPMO_SYNCPOINT`). Messages survive broker restart.
- **RocketMQ:** Use `SYNC_FLUSH` mode — broker writes to disk synchronously before returning ACK.

Consumer side — don't lose the message after receiving it:

- **Do not commit the offset/ACK before processing is complete.** Process first, ACK last.
- If the consumer crashes mid-processing, the broker re-delivers the message (this is the cause of duplicate consumption — see Q10-2).

中文解释：

可靠投递分三段保证：

生产者侧 — 消息不丢失在发送途中：

1. **同步发送 + 重试**： `send()` 后等待Broker的ACK，超时或失败则指数退避重试。注意：重试可能导致重复（这是Q10-2的话题）。
2. **事务性发件箱模式（Transactional Outbox）**：业务数据和消息一起写入同一数据库事务（outbox表），另起一个relay进程从outbox表读取并发送到MQ。彻底消除"数据库提交成功但消息丢失"的问题，是生产级推荐方案。

Broker侧 — 消息不丢失在Broker内：

- **Kafka**： `acks=all`， `replication.factor=3`， `min.insync.replicas=2`，消息写入所有ISR副本后才ACK。
- **IBM-MQ**：使用持久化消息（PERSISTENT），消息写到磁盘，Broker重启后消息仍在。
- **RocketMQ**： `SYNC_FLUSH` 模式，Broker同步刷盘后才返回ACK（代价是写入延迟略高）。

消费者侧 — 消息不丢失在处理中：

- **先处理，后提交offset/ACK**。不能先提交再处理——一旦提交后服务崩溃，消息就永久丢失了。
- 先处理后提交的代价：崩溃重启后消息会被重新投递，产生重复消费（Q10-2处理）。

面试口诀：生产者用Outbox，Broker开持久化+多副本，消费者先处理后ACK。

Q10-2. How do you handle duplicate message consumption (idempotency)?

问题10-2：如何处理重复消费？（幂等性设计）

English Answer:

Duplicate messages are **unavoidable** in any distributed system. The network can fail after the broker sends a message but before the consumer ACKs it — so the broker sends it again. The correct solution is to make your consumer **idempotent**: processing the same message twice has the same result as processing it once.

Three common patterns:

1. Database unique constraint (most reliable for financial systems):

Every message has a unique `messageId` . Before inserting the result, check if a record with that `messageId` already exists. Use a unique index to prevent race conditions.

```
// Deduplicate using a processed_events table
@Transactional
public void handleOrderPlaced(OrderPlacedEvent event) {
    if (processedEventRepository.existsById(event.getMessageId())) {
        log.info("Duplicate message, skipping: {}", event.getMessageId());
        return; // idempotent – safe to skip
    }
    // Process the business logic
    portfolioService.updateHolding(event.getOrderId());
    // Mark as processed in same transaction
    processedEventRepository.save(new ProcessedEvent(event.getMessageId()));
}
```

2. Idempotent business operation (natural idempotency):

Design the operation so it is safe to run multiple times. For example, instead of `balance += 100` (not idempotent), use `SET balance = 100 WHERE balance < 100 AND version = 5` (idempotent with optimistic lock). A `status = 'PAID'` update is also idempotent — setting it twice has no effect.

3. Redis deduplication (high throughput, short TTL):

```
String key = "msg:processed:" + event.getMessageId();
Boolean isNew = redisTemplate.opsForValue().setIfAbsent(key, "1",
    Duration.ofHours(24));
if (Boolean.FALSE.equals(isNew)) {
    return; // already processed
}
// proceed with business logic
```

When to use which:

Pattern	Pros	Cons
DB unique constraint	Durable, 100% reliable	Extra DB write per message
Natural idempotency	No extra storage	Requires careful design
Redis dedup	Very fast, low DB pressure	Redis data can be lost, has TTL

For financial transactions (payment, trade execution), always use the DB unique constraint approach. Redis is acceptable for lower-stakes operations like analytics events.

中文解释：

重复消费在分布式系统中**无法完全避免**（网络分区、消费者崩溃都会导致消息重投），正确做法是让消费者**幂等**。

三种主流方案：

- 1. 数据库唯一约束（金融系统首选）：** 每条消息携带全局唯一的 `messageId`，消费前查 `processed_events` 表，已处理则跳过。用数据库唯一索引防止并发竞争。消息处理和写入 `processed_events` 放同一个事务，保证原子性。
- 2. 业务天然幂等（最优雅）：** 将操作设计成幂等的。例如：`UPDATE order SET status='PAID' WHERE id=? AND status='PENDING'`（重复执行没有副作用）；使用乐观锁版本号控制更新，天然防重。金融对账、状态机流转通常可以这样设计。
- 3. Redis去重（高吞吐量场景）：** 用 `messageId` 作为key做 `SET NX EX`，已存在则跳过。缺点：Redis持久化配置不当可能丢去重记录；设置TTL后超期会"解禁"重复（所以不适合金融转账）。

面试答法核心： 幂等性是消费者自己的责任，不能依赖MQ只投一次。`at-least-once` 是所有主流MQ的默认语义，必须在业务层做幂等处理。

Q10-3. How do you guarantee message ordering? What are the trade-offs?

问题10-3：如何保证消息顺序？有哪些权衡取舍？

English Answer:

Message ordering is one of the hardest problems in distributed messaging. Let's break it down.

Why ordering is lost:

- Multiple partitions/queues: Kafka splits a topic into partitions. Producer sends messages to different partitions in parallel — order is not guaranteed across partitions.
- Multiple consumers: If multiple consumer threads or instances process messages concurrently, they may process out of order even from the same partition.
- Retry: A failed message gets retried later — it arrives "after" messages that were sent after it.

Solution 1 — Partition by business key (Kafka best practice):

All messages for the same "entity" (same order ID, same user ID) must go to the **same partition**. Within a partition, Kafka guarantees order.

```
// Producer – use orderId as the partition key
kafkaTemplate.send("order-events", order.getId().toString(), event);
// All events for the same order go to the same partition, in order
```

Solution 2 — Single queue per entity (IBM-MQ / RocketMQ):

Use a separate queue or message group for each entity. IBM-MQ supports **message grouping** — messages in the same group are delivered in order to one consumer. RocketMQ has **Message Orderly** — one consumer thread per queue.

```
// RocketMQ – consume in order
@RocketMQMessageListener(
    topic = "order-topic",
    consumerGroup = "order-group",
    consumeMode = ConsumeMode.ORDERLY // single-threaded per queue
)
public class OrderConsumer implements RocketMQListener<OrderEvent> { ... }
```

Solution 3 — Sequence number + reorder buffer (complex, use when unavoidable):

Add a sequence number to each message. Consumer buffers out-of-order messages and processes them in sequence. High complexity — avoid if possible.

Trade-offs:

Approach	Ordering guarantee	Throughput impact
Partition by key	Per-entity order	Good (parallel partitions)
Single queue/thread	Global order	Low (bottleneck)
Reorder buffer	Flexible	High complexity

中文解释：

顺序问题的根源：

- 多分区并行：Kafka一个topic有多个partition，不同partition间无顺序保证。
- 多消费者并发：多个线程/实例同时处理消息，即使消息有序到达也可能乱序处理。

- 重试：失败消息重试后"插队"到比它晚发出的消息之后。

解决方案1 — 按业务Key路由到同一分区（Kafka推荐）： 将 `orderId / userId` 作为消息的 partition key，同一实体的所有消息路由到同一partition。Kafka保证单partition内有序，多partition间并发处理提升吞吐量。适用于99%的业务顺序场景（只需保证同一实体的消息有序，不需要全局顺序）。

解决方案2 — 消息分组/顺序消费（IBM-MQ / RocketMQ）： IBM-MQ的Message Group，同一Group内消息按顺序投递给同一个消费者。RocketMQ的 `ORDERLY` 消费模式，每个queue分配一个消费线程，保证queue内顺序。

关键权衡：

- 全局顺序（所有消息都有序）= 只能用一个queue/partition = 吞吐量瓶颈，生产上几乎不用。
- 分区顺序（同Key消息有序）= 多partition并行 = 满足绝大多数业务需求，**推荐**。

面试陷阱： 不要说"用单线程消费保证顺序"——这是以牺牲吞吐量换顺序，面试官会追问"那如果QPS很高怎么办"。正确答案是分区顺序 + 按业务Key路由。

Q10-4. How do you handle a message backlog (messages piling up faster than they are consumed)?

问题10-4：如何处理消息积压问题？

English Answer:

A message backlog happens when producers send faster than consumers can process. This is a serious incident — in a financial system it means delayed trades, slow order processing, and angry users.

Step 1 — Diagnose first:

- Is this a sudden spike in producers (e.g., market open, batch job started)?
- Is this a consumer slowdown (e.g., downstream database is slow, a new code deployment introduced a bug)?
- What is the consumer lag in each partition/queue?

```
# Kafka – check consumer group lag
kafka-consumer-groups.sh --bootstrap-server kafka:9092 \
  --group order-consumer-group --describe
# Look at LAG column – how many messages are behind
```

Step 2 — Emergency relief (scale up consumers):

- **Add more consumer instances** (if stateless and idempotent). For Kafka: add more consumer instances up to the number of partitions. For IBM-MQ: add more competing consumers.
- **Increase consumer thread count** in the short term.
- If partitions are the bottleneck (all instances assigned, still slow): **increase partition count** (for Kafka). Note: this is a structural change, plan carefully.

Step 3 — Fix the root cause:

- If consumers are slow due to a database bottleneck: add a read replica, add cache (Redis), or batch-process instead of one-by-one.
- If a code bug caused consumer crash loops: fix the bug and redeploy.
- If producers legitimately increased: permanent capacity increase — more partitions, more consumer instances.

Step 4 — Prevent future backlogs:

- Set up **consumer lag alerts** (alert when lag > threshold for > 5 minutes).
- Use **rate limiting on producers** for known bursty sources (like market data feeds).
- Keep consumer processing time as short as possible — do the minimal work in the consumer, push heavy work to async threads.

中文解释：

消息积压是生产事故，需要快速诊断和响应。

第一步 — 诊断原因：

- 是生产者突增（开盘、批处理任务）？还是消费者变慢（下游DB慢、新版本代码Bug）？
- 用监控看消费者Lag趋势：是突然升高还是持续增长？
- Kafka： `kafka-consumer-groups.sh --describe` 查看各partition的LAG值。

第二步 — 紧急扩容消费者：

- 增加消费者实例数（Kafka中消费者数不能超过partition数，超出的实例会空闲）。

- 临时增大单实例的消费并发线程数（注意幂等性要能支撑并发）。
- 如果partition数成为上限：增加partition数（不可逆操作，需谨慎规划）。

第三步 — 修复根因：

- 消费者慢 → 查下游DB（加只读副本、加Redis缓存）、查GC（内存泄漏）、查网络。
- 代码Bug导致消费者崩溃重启 → 修Bug回滚或热修复。
- 生产者流量永久性增长 → 扩容partition + 消费者实例。

第四步 — 预防：

- 监控Lag，设置告警（Lag > N 且持续 > 5分钟触发告警）。
- 对已知突发流量场景（开盘、活动）提前扩容。
- 消费者逻辑尽量轻量，重处理移到异步线程池，不阻塞消费loop。

面试答法：分四步——诊断 → 紧急扩容消费者 → 修复根因 → 预防监控。不要只说"加消费者"，面试官想看到你有完整的incident处理思路。

Q10-5. What is a Dead Letter Queue (DLQ)? How do you design a DLQ strategy?

问题10-5：什么是死信队列（DLQ）？如何设计死信队列策略？

English Answer:

A **Dead Letter Queue (DLQ)** is a special queue where messages go when they **cannot be processed successfully** after a maximum number of retries. Instead of blocking the main queue or dropping the message, the system routes the "poison pill" message to the DLQ for investigation.

Why DLQ matters:

Without a DLQ, a bad message (malformed payload, unexpected data) will keep failing, block the consumer, and stop all subsequent messages from being processed — this is called a "poison pill" problem.

DLQ design — three decisions:

1. Retry policy before DLQ:

```
# Spring Kafka retry configuration
spring.kafka.listener.retry.max-attempts: 3
spring.kafka.listener.retry.backoff.initial-interval: 1000ms
spring.kafka.listener.retry.backoff.multiplier: 2.0
# Sequence: attempt 1 → wait 1s → attempt 2 → wait 2s → attempt 3 → DLQ
```

2. DLQ naming convention: Use a consistent pattern so DLQ messages are easy to find:

- Kafka: `order-events.DLQ` or `order-events-dlq`
- IBM-MQ: `DEAD.LETTER.QUEUE` (system-level) or `APP.ORDER.DLQ` (application-level)

3. DLQ message enrichment: Add context to the DLQ message so engineers can debug it:

```
public record DlqMessage(
    String originalTopic,
    String originalPayload,
    String errorMessage,
    String stackTrace,
    int retryCount,
    Instant failedAt
) {}
```

DLQ operations:

- **Monitoring:** Alert when DLQ message count > 0 (every DLQ message is a bug or data issue).
- **Replay:** After fixing the root cause, replay DLQ messages back to the original topic.
- **Manual inspection:** Build a simple admin tool to view DLQ messages without special access to the broker.

```
// Example: replay DLQ messages back to original topic
@Scheduled(cron = "0 0 3 * * *") // 3am nightly
public void replayDlqMessages() {
    List<DlqMessage> failed = dlqRepository.findAll();
    failed.forEach(msg -> {
        kafkaTemplate.send(msg.originalTopic(), msg.originalPayload());
        dlqRepository.delete(msg);
    });
}
```

中文解释:

死信队列（DLQ）： 消息经过N次重试仍然处理失败后，被路由到一个特殊的队列（DLQ），而不是丢弃或无限重试阻塞主队列。

为什么需要DLQ： 没有DLQ的情况下，一条"毒消息"（格式错误、依赖数据不存在）会让消费者陷入无限重试循环，后续正常消息全部被阻塞——这就是"毒丸问题"（poison pill）。

DLQ设计的三个决策：

1. 重试策略： 定义重试次数和退避间隔。典型配置：最多3次，间隔1s/2s/4s（指数退避），3次失败后进DLQ。

2. DLQ命名规范： 原topic名.DLQ 或 原topic名-dlq，一目了然。每个业务topic对应一个DLQ，不要共用。

3. DLQ消息信息增强： 进入DLQ时附加错误信息（原始Payload、异常堆栈、重试次数、失败时间），方便排查。否则只有原始消息，不知道为什么失败。

DLQ运营三件事：

- **监控告警：** DLQ有消息 = 必须告警，每一条DLQ消息都是Bug或数据问题。
- **重放（Replay）：** 修复根因后，将DLQ消息重新投递回原topic重新消费。
- **可视化：** 提供简单的管理界面查看DLQ消息内容，不需要运维人员直接登录Broker。

面试答法： DLQ是消息可靠性的"最后防线"，设计时关注三点：重试策略、消息元数据增强、重放机制。监控上DLQ消息数必须是关键告警指标。

Q10-6. Technology Selection: IBM-MQ vs Kafka vs RocketMQ — When to use which?

问题10-6：技术选型 — IBM-MQ、Kafka、RocketMQ 怎么选？

English Answer:

There is no single "best" MQ. The right choice depends on your requirements. Here is a structured comparison.

Core characteristics:

Dimension	IBM-MQ	Apache Kafka	Apache RocketMQ
Primary model	Point-to-point queue	Event streaming / pub-sub log	Hybrid (queue + streaming)
Message retention	Consumed then deleted	Retained for days/weeks (replay)	Retained, configurable
Ordering	FIFO per queue, message groups	Per-partition ordered	Ordered queues, global order option
Throughput	Medium (thousands/sec)	Very high (millions/sec per broker)	High (hundreds of thousands/sec)
Exactly-once	Yes (XA transactions)	Kafka transactions (complex)	Transactional messages
Ecosystem	JMS, MQ .NET, mainframe	Kafka Streams, Kafka Connect, Schema Registry	Spring integration, Alibaba ecosystem
License & support	IBM commercial license, \$\$\$	Apache open source / Confluent paid	Apache open source / Alibaba Cloud
Operational complexity	Low (IBM managed)	High (ZooKeeper/KRaft, tuning)	Medium

Decision guide — use IBM-MQ when:

- Your organization already has IBM-MQ infrastructure and IBM enterprise support contracts.
- You need to integrate with **mainframe systems** or legacy COBOL applications (IBM-MQ is the standard in banking for this).
- You need **XA transactions** — guaranteed exactly-once across MQ + database in a two-phase commit.
- Compliance requires enterprise-grade SLA and certified disaster recovery.

Decision guide — use Kafka when:

- You need **very high throughput** (hundreds of thousands to millions of messages per second).
- Multiple downstream services need to **independently consume the same event** (e.g., risk engine, audit, analytics all read the same trade event).
- You need **event replay** — replay all events from the last 7 days to rebuild a new service's state.

- You are building a **real-time data pipeline** — CDC, streaming analytics, ETL.
- Your team is cloud-native and familiar with Kafka's operational model.

Decision guide — use RocketMQ when:

- Your team is in the **Alibaba/Chinese tech ecosystem** and needs Alibaba Cloud integration.
- You need a **transactional message** pattern (half-message + local transaction + confirm) that is simpler than Kafka transactions.
- You need **delay messages** natively (send a message to be delivered in 30 seconds — Kafka needs custom implementation, RocketMQ has it built-in).
- You need a balance between Kafka's high throughput and IBM-MQ's reliability, with simpler operations.

RocketMQ's unique killer feature — Transactional Messages:

1. Producer sends a "half message" (invisible to consumers)
2. Producer executes local database transaction
3. If DB commit succeeds → producer sends COMMIT to broker → message becomes visible
4. If DB commit fails → producer sends ROLLBACK → message discarded
5. If producer crashes → broker queries producer to confirm (transaction check)

This solves the distributed transaction problem without XA — simpler and more performant than two-phase commit.

Summary for interviews:

- **IBM-MQ**: Legacy banking, mainframe integration, XA transactions, enterprise compliance.
- **Kafka**: High throughput, event streaming, multi-subscriber fan-out, event sourcing, data pipelines.
- **RocketMQ**: Transactional messages, delay queues, Alibaba ecosystem, simpler ops than Kafka.

In practice, most large financial systems use **IBM-MQ for critical transactional commands** and **Kafka for event broadcasting and analytics** — they coexist.

中文解释:

三者核心定位:

维度	IBM-MQ	Kafka	RocketMQ
核心模型	点对点队列	事件流/持久化日志	队列 + 流，混合模型
消息保留	消费后删除	按时间保留（可回放）	可配置保留时间
吞吐量	中等（千级/秒）	极高（百万级/秒）	高（十万级/秒）
精确一次	XA事务（强支持）	Kafka事务（复杂）	事务消息（简洁）
延迟消息	不支持	不原生支持（需自实现）	原生支持
运维复杂度	低（IBM托管）	高（需要ZooKeeper/KRaft调优）	中
许可/费用	IBM商业授权（贵）	开源/Confluent商业版	开源/阿里云版

选型决策树：

选 IBM-MQ 的场景：

- 公司已有IBM-MQ基础设施和IBM企业支持合同（迁移成本高，维持现状合理）。
- 需要与大型机（Mainframe）或 COBOL系统集成（IBM-MQ是银行核心系统的标准协议）。
- 需要XA事务（MQ + 数据库的两阶段提交，保证精确一次）。
- 监管合规要求企业级SLA和经认证的灾备方案。

选 Kafka 的场景：

- 需要超高吞吐量（十万至百万级TPS）。
- 多个下游服务需要独立消费同一事件（风控、审计、分析分别订阅交易事件）。
- 需要事件回放（新服务上线，需要消费历史7天数据重建状态）。
- 构建实时数据管道（CDC数据同步、流式计算、ETL）。

选 RocketMQ 的场景：

- 团队在阿里/国内互联网生态中，需要与阿里云深度集成。
- 需要延迟消息原生支持（30秒后投递，无需自己实现）。
- 需要事务消息（半消息机制，比Kafka事务简单得多）。
- 介于Kafka吞吐量和IBM-MQ可靠性之间，且运维能力相对有限。

RocketMQ事务消息原理（面试加分项）：

1. 生产者发送"半消息"（消费者不可见）

2. 生产者执行本地数据库事务

3. DB提交成功 → 发送COMMIT → 消息对消费者可见

4. DB提交失败 → 发送ROLLBACK → 消息丢弃

5. 生产者崩溃 → Broker回查生产者确认事务状态

这个模式解决了分布式事务问题，不需要XA，比两阶段提交简单且性能更好。

面试标准答法：三者不互斥，大型金融系统常见的是**IBM-MQ处理核心交易命令**（下单、转账）+ **Kafka做事件广播和数据管道**共存。RocketMQ在阿里生态或需要延迟消息时是Kafka的好替代。

Quick Reference — New JD Key Technology Choices

新 JD 核心技术选型速查

Technology	Key Point for Interview
Spring Boot 3	Jakarta EE, Java 17+, Micrometer Tracing
Spring Cloud	OpenFeign, Resilience4j, Config Server, Eureka
Spring Batch	Chunk processing, restartable, for large data jobs
Hibernate L1/L2	L1=session-scoped, L2=application-scoped, old data risk
React.js	useState (state), useEffect (extra actions), React.memo (perf)
Context API	Cross-component state without prop drilling
Docker multi-stage	Build stage (Maven+JDK) → Runtime stage (JRE only)
JUnit 5	assertThrows, assertDoesNotThrow, assertAll
@Mock vs @MockBean	@Mock=unit test (no Spring), @MockBean=integration test
Scrum vs Kanban	Scrum=fixed sprints (dev), Kanban=continuous flow (ops)